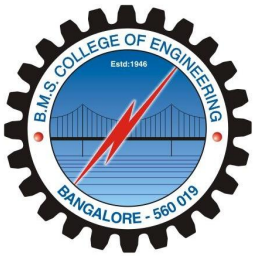
# Course – Artificial Intelligence



# Unit-2

**Uninformed Search Strategies**: Breadth First search, Depth First Search, Iterative deepening depth first search, Hill Climbing, Simulated annealing and Monotonicity; **Informed Search Strategies**: Heuristic functions, Greedy best first search, A*search. Heuristic Functions.

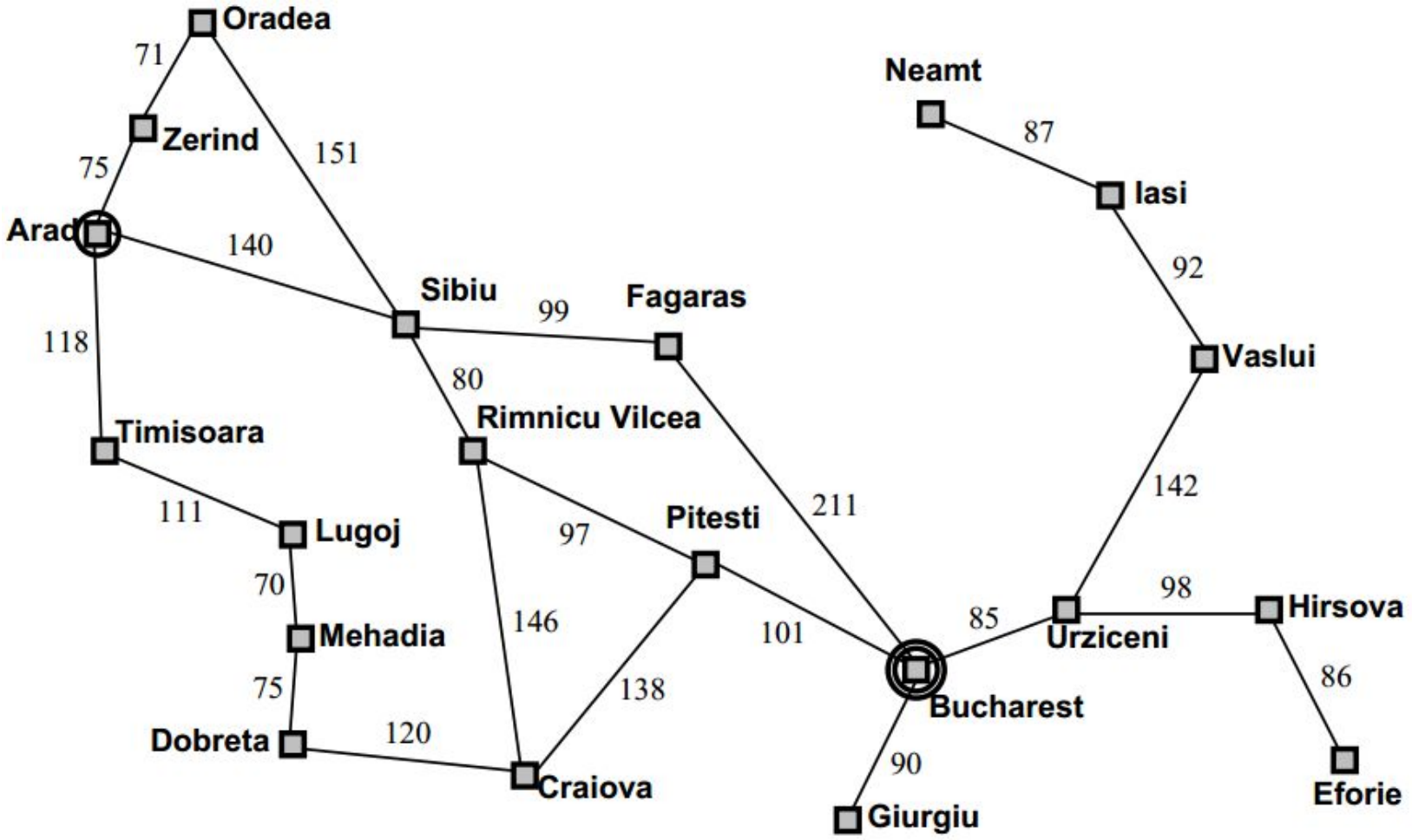
# Searching for Solution (Unit-1 Recap)

A solution is an action sequence, so search algorithms work by considering various possible action sequences.

The possible action sequences starting at the initial state form a **search tree** with the initial state at the root

The branches are actions and the **nodes** correspond to states in the state space of the problem.

# Example – Romania Route Map (Unit-1 Recap)



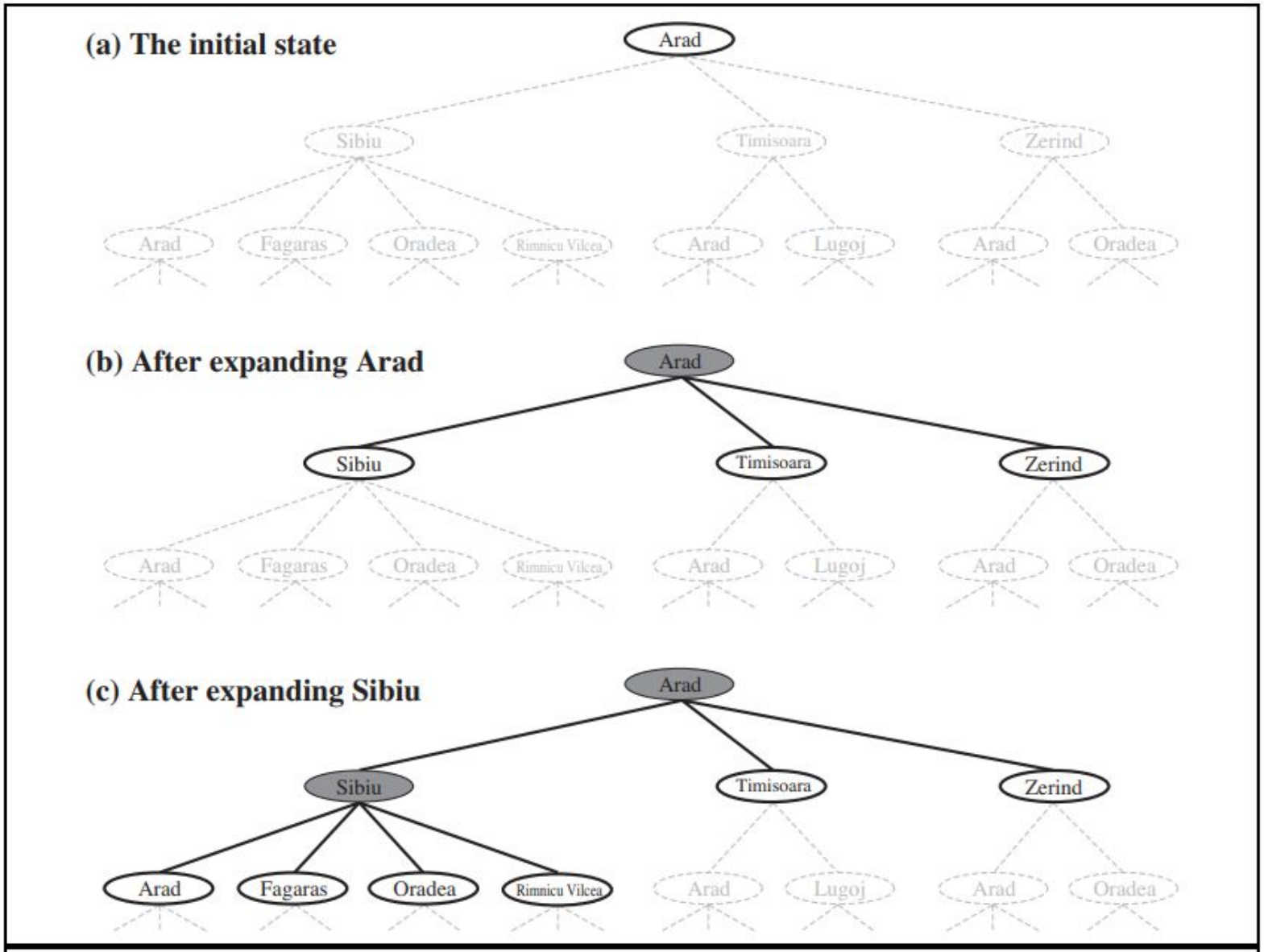



**Figure** Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

# Frontier (Unit-1 Recap)

We reach a state when we identify a path from the start state to it. But, we say that we expanded it if we had followed all its outward edges and reached all its children.

So, we can also think of a search as a sequence of expansions, and we first have to reach a state before expanding it.

**Frontier is the reached but unexpanded states because we can expand only them**

# (Unit-1 Recap)

The root node of the tree corresponds to the initial state, *In(Arad)*.

The first step is to test whether this is a goal state.

Then we need to consider taking various actions. We do this by **expanding** the current state;

applying each legal action to the current state, thereby **generating** a new set of states.

In this case, we add three branches from the **parent node** *In(Arad)* leading to three new **child nodes**: *In(Sibiu)*, *In(Timisoara)*, and *In(Zerind)*.

Now we must choose which of these three possibilities to consider further.

The process of expanding nodes on the frontier continues until either a solution is found or there are no more states to expand.

**search strategy**: how they choose which state to expand next

# General tree-search and graph-search algorithms (Unit-1 Recap)

**function** TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  **loop do**
    **if** the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    **if** the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

---

**function** GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  ***initialize the explored set to be empty***
  **loop do**
    **if** the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    **if** the node contains a goal state **then return** the corresponding solution
    ***add the node to the explored set***
    expand the chosen node, adding the resulting nodes to the frontier
      ***only if not in the frontier or explored set***

**Figure** [ ] An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

# Tree Search (TS) Algorithms
## (Unit-1 Recap)

```
function TREE-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

# Graph Search (GS) Algorithms
## (Unit-1 Recap)

```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

# General tree-search and graph-search algorithms (Unit-1 Recap)

**function** TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  **loop do**
    **if** the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    **if** the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

---

**function** GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  ***initialize the explored set to be empty***
  **loop do**
    **if** the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    **if** the node contains a goal state **then return** the corresponding solution
    ***add the node to the explored set***
    expand the chosen node, adding the resulting nodes to the frontier
      ***only if not in the frontier or explored set***

**Figure** An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.

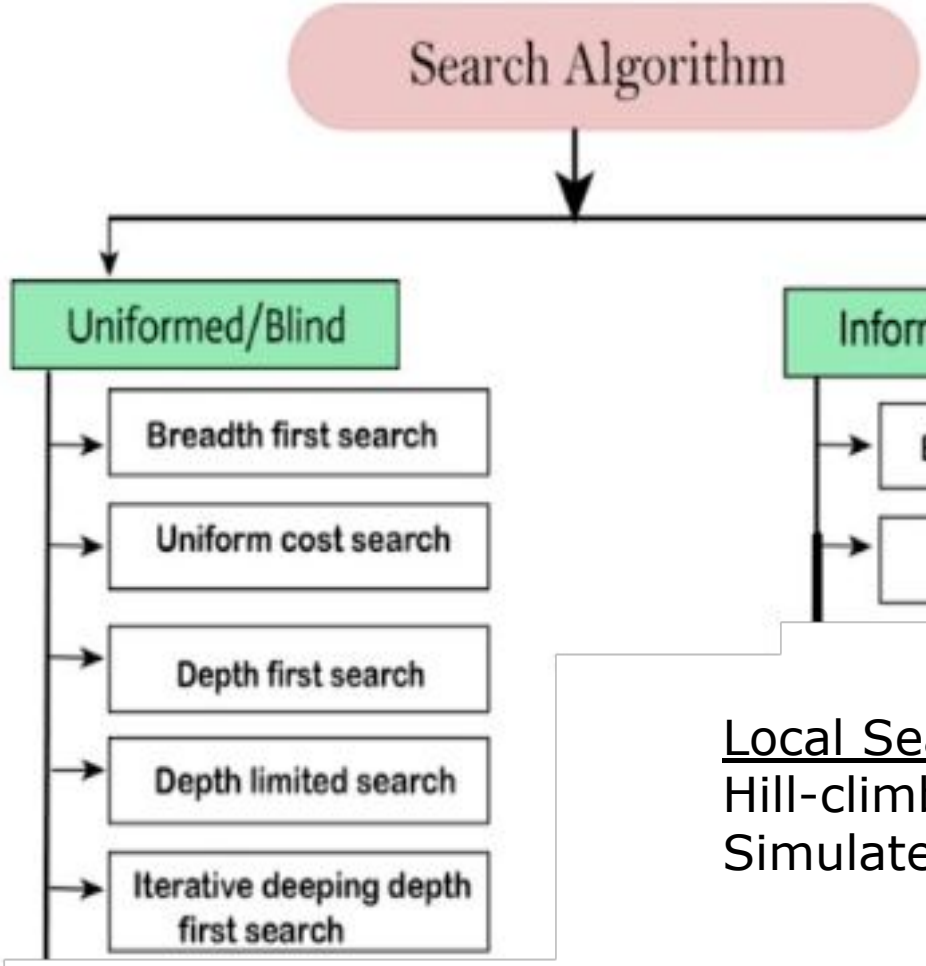
# Difference between TS and GS
## (Unit-1 Recap)

- The difference is not whether the problem is a tree or a general graph!
- The distinction is, instead, how we are traversing the search space (represented as a graph) to search for our goal state and whether we are using an additional list (called the closed list) or not.

| Tree Search | Graph Search |
|---|---|
| In the case of a tree search, we do **not** keep this closed list. Consequently, the same node can be visited multiple (or even infinitely many) times, which means that the produced tree (by the tree search) may contain the same node multiple times. | In the case of a graph search, we use a list, called the **closed list** (also called **explored set**), to keep track of the nodes that have already been visited and expanded, so that they are not visited and expanded again |

# Difference between TS and GS
## (Unit-1 Recap)

| Tree Search | Graph Search |
|---|---|
| In the case of a tree search, we do **not** keep this closed list. Consequently, the same node can be visited multiple (or even infinitely many) times, which means that the produced tree (by the tree search) may contain the same node multiple times. | In the case of a graph search, we use a list, called the **closed list** (also called **explored set**), to keep track of the nodes that have already been visited and expanded, so that they are not visited and expanded again |
| **Advantage:** Can visit the same node multiple times. | **Advantage:** If we finish the search of a node, we will never search it again. |
| | **Disadvantage:** Graph search uses more memory (which we may or may not have) than tree search. This matters because graph search actually has *exponential* memory requirements in the worst case. |




Local Search Algorithms
Hill-climbing search
Simulated Annealing

# Search Strategies

1. Uniformed or Blind Search: Uninformed Search Strategies have **no additional** information about states beyond that provided in the problem definition.

2. Informed or Heuristic Search: Search Strategies that know whether one non goal state is "more promising" than another are called Informed or Heuristic Search

# Uninformed vs Informed Search

| Feature | Uninformed Search | Informed Search |
|---|---|---|
| Extra Info Used? | ❌ No | ✅ Yes (heuristics) |
| Speed | Slow | Faster |
| Efficiency | Tries many paths | Tries fewer, better paths |
| Real-life Example (Searching classroom) | Checking all rooms randomly | Checking the signs next to the doors |

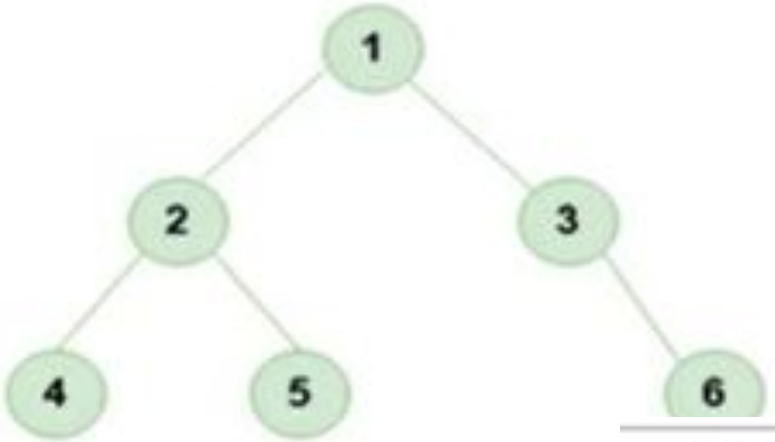
**Remember:**

- Use Uninformed Search when no extra info is available.
- Use Informed Search when helpful info is available.

# Breadth First search

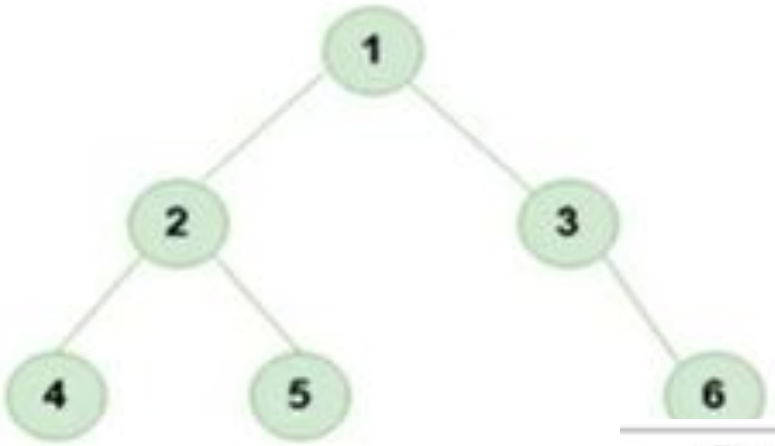
- The root node is expanded first and then all the successors of the nodes are expended next, and their successors and so on.
- In general, all the nodes at a given depth are expanded in the search tree before any node at the next level are expanded.
  - Mark any node as start (initial) node
  - Explore and traverse un-visited nodes adjacent to starting node
  - Mark nodes as completed and move to next adjacent and un-visited node.

# BFS Example




| Step | Queue | Visited Nodes |
|---|---|---|
| 1 | [1] | {} |
| 2 | [2, 3] | {1} |
| 3 | [3, 4, 5] | {1, 2} |
| 4 | [4, 5, 6] | {1, 2, 3} |
| 5 | [5, 6] | {1, 2, 3, 4} |
| 6 | [6] | {1,2,3,4,5} |
| 7 | [] | {1,2,3,4,5,6} |

# BFS Example



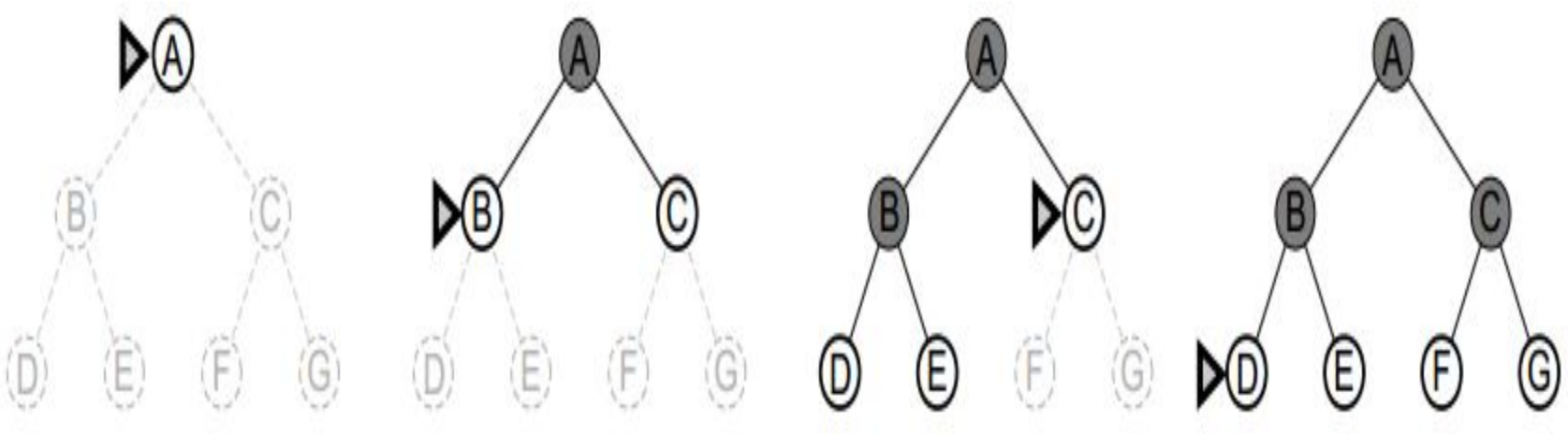

| Step | Queue | Visited Nodes |
|---|---|---|
| 1 | [1] | {} |
| 2 | [2, 3] | {1} |
| 3 | [3, 4, 5] | {1, 2} |
| 4 | [4, 5, 6] | {1, 2, 3} |
| 5 | [5, 6] | {1, 2, 3, 4} |
| 6 | [6] | {1,2,3,4,5} |
| 7 | [] | {1,2,3,4,5,6} |

Real-life Example:
Like trying to find your way in a city by checking every road one-by-one.
You will find the shortest path, but it will take time and you will need a big notebook to remember all roads you already checked.

# Breadth-First Search, Illustration

- Nodes are expanded in the order they were produced

(frontier ← a FIFO queue).



# Breadth-First Search, Illustration

## Algorithm

**Breadth first search**

Let *fringe* be a list containing the initial state

Loop

if *fringe* is empty return failure

Node ← remove-first (fringe)

if Node is a goal

then return the path from initial state to Node

else generate all successors of Node, and

(merge the newly generated nodes into *fringe*)

add generated nodes to the back of *fringe*

End Loop

# Breadth-First Search - Complexity

Time Complexity:

- Let b be the maximal branching factor and d the depth of a solution path. Then the maximal number of nodes expanded is

  $1+b + b^2 + b^3 + \cdots + b^d + (b^{d+1} - b)$

  $1+b + b^2 + b^3 + \cdots + b^d + b(b^d - 1) \in O(b^{d+1})$

Space Complexity:

- Every node generated is kept in memory.
- $O(b^{d+1})$

# Breadth-First Search: Optimal and Complete

- Breadth-first search is an instance of the general graph-search algorithm in which the shallowest unexpanded node is chosen for expansion. This is achieved very simply by using a FIFO queue for the frontier. Thus, new nodes (which are always deeper than their parents) go to the back of the queue, and old nodes, which are shallower than the new nodes, get expanded first.
- BFS is complete—if the shallowest goal node is at some finite depth d, breadth-first search will eventually find it after generating all shallower nodes (provided the branching factor b is finite). Note that as soon as a goal node is generated, we know it is the shallowest goal node because all shallower nodes must have been generated already and failed the goal test. Now, the shallowest goal node is not necessarily the optimal one; technically, breadth-first search is optimal if the path cost is a nondecreasing function of the depth of the node. The most common such scenario is that all actions have the same cost.

# Breadth-First Search, Example Data

Example: b = 10; 10, 000 nodes/second;
1, 000 bytes/node

| Depth | Nodes | Time | Memory |
|---|---|---|---|
| 2 | 110 | .11 milliseconds | 107 kilobytes |
| 4 | 11,110 | 11 milliseconds | 10.6 megabytes |
| 6 | $10^6$ | 1.1 seconds | 1 gigabyte |
| 8 | $10^8$ | 2 minutes | 103 gigabytes |
| 10 | $10^{10}$ | 3 hours | 10 terabytes |
| 12 | $10^{12}$ | 13 days | 1 petabyte |
| 14 | $10^{14}$ | 3.5 years | 99 petabytes |
| 16 | $10^{16}$ | 350 years | 10 exabytes |

Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

So, which is the worse problem, time or memory?

# Advantages and Disadvantages of BFS

| Advantages | Disadvantages |
| --- | --- |
| Finds the shortest path (if all steps cost the same). Which means it is always optimal. | Uses a lot of memory (stores many nodes) |
| Very simple and easy to understand | Can be slow for deep or big problems |
| Always finds a solution if one exists (complete method) | Doesn't work well when paths are very long |
| Works well for puzzles and simple maps | Not suitable for real-time or memory-limited devices |

# Uniform-cost search

- Use a priority queue instead of a simple queue
- Insert nodes in the increasing order of the cost of the path so far
- Guaranteed to find an optimal solution!

Enqueue nodes by path cost.

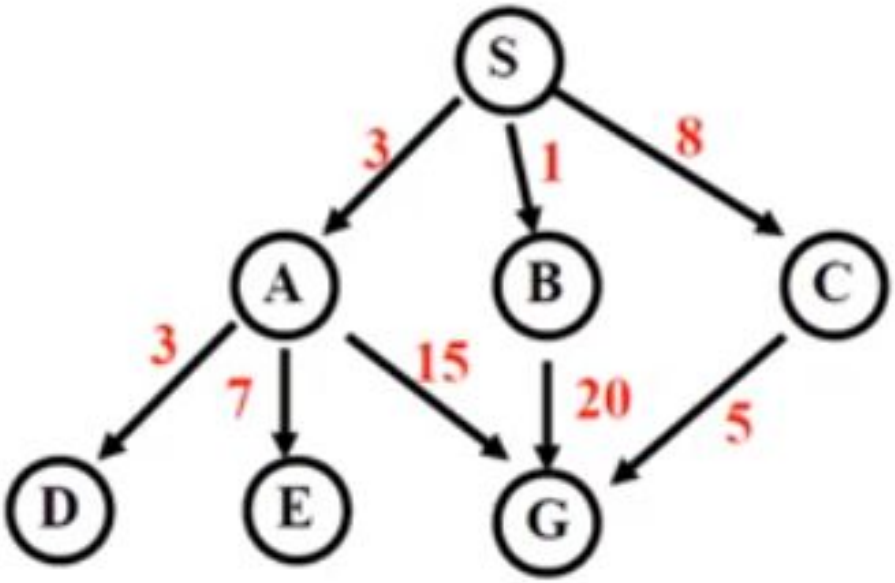
That is, let g(n) = cost of the path from the start node to the current node n.

Sort nodes by increasing value of g.

Uniform-cost search expands the node n with the lowest path cost g(n)

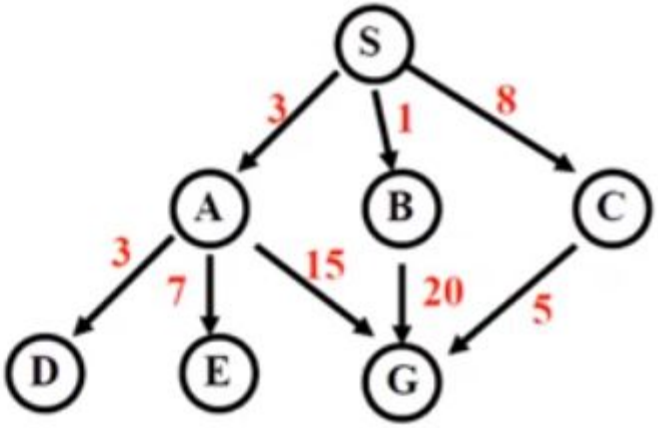
# Problem

Apply Uniform cost search to find path from S to G

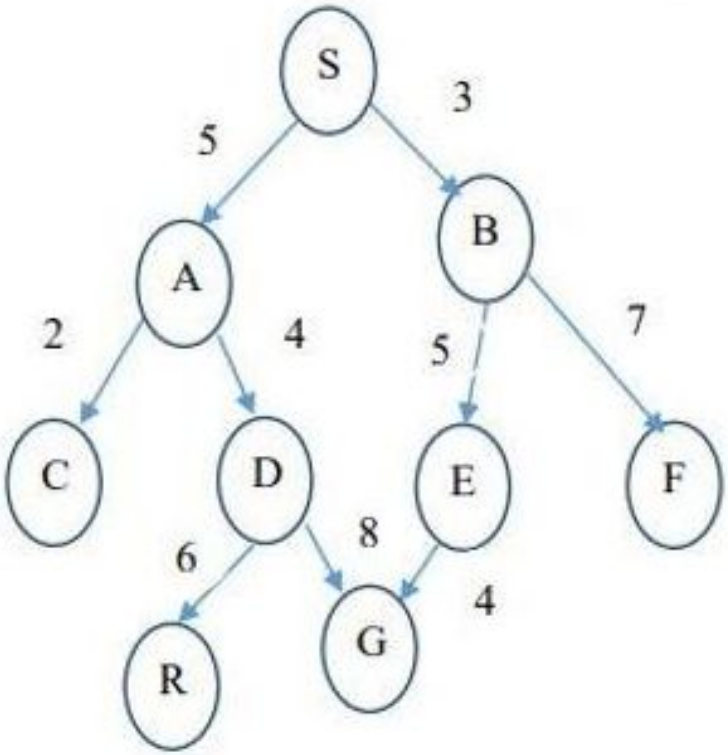


# Solution

- Expanded node Nodes list
- { S0 }
- S0 { B1, A3, C8 }
- B1 { A3, C8, G21 }
- A3 { D6, C8, E10, G18, G21 }
- D6 { C8, E10, G18, G21 }
- C8 { E10, G13, G18, G21 }
- E10 { G13, G18, G21 }
- G13 { G18, G21 }
- Solution path found is S C G, cost 13
- Number of nodes expanded (including goal node) = 7



Perform the Uniform Cost Search (UCS) algorithm on the following graph with path costs (Initial node is S and Goal node is G)



# Uniform Cost Search

Time Complexity

Let C* is Cost of the optimal solution, and ε is each step to get closer to the goal node. Then the number of steps is = [C*/ε]+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is$O(b^{1 + [C*/\varepsilon]})$.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + [C*/\varepsilon]})$.

# Uniform-cost search is optimal

Because,

i. Whenever uniform-cost search selects a node n for expansion, the optimal path to that node has been found.

ii. Step costs are nonnegative, paths never get shorter as nodes are added.

These two facts together imply that uniform-cost search expands nodes in order of their optimal path cost. Hence, the first goal node selected for expansion must be the optimal solution.

Note: Uniform-cost search does not care about the number of steps a path has, but only about their total cost. Therefore, it will get stuck in an infinite loop if there is a path with an infinite sequence of zero-cost actions

# Uniform-cost search is Complete

- Uniform-cost search does not care about the number of steps a path has, but only about their total cost. Therefore, **it will get stuck in an infinite loop if there is a path with an infinite sequence of zero-cost actions.** Completeness is guaranteed provided the cost of every step exceeds some small positive constant.

# Time complexity of Uniform Cost Search

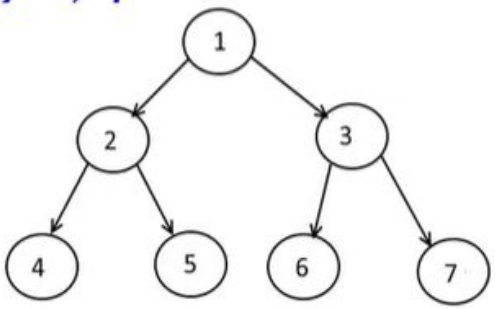
Uniform-cost search is guided by path costs rather than depths, so its complexity is not easily characterized in terms of $b$ and $d$. Instead, let $C^*$ be the cost of the optimal solution, and assume that every action costs at least $\epsilon$. Then the algorithm's worst-case time and space complexity is $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, which can be much greater than $b^d$. This is because uniform-cost search can explore large trees of small steps before exploring paths involving large and perhaps useful steps. When all step costs are equal, $b^{1+\lfloor C^*/\epsilon \rfloor}$ is just $b^{d+1}$. When all step costs are the same, uniform-cost search is similar to breadth-first search, except that the latter stops as soon as it generates a goal, whereas uniform-cost search examines all the nodes at the goal's depth to see if one has a lower cost; thus uniform-cost search does strictly more work by expanding nodes at depth $d$ unnecessarily.

# Uniform cost search on a Graph

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

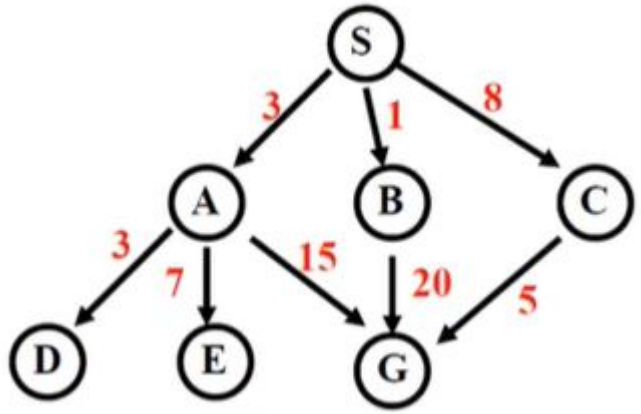
# Depth First Search

- Start with the root node and completely explore the left most child nodes, before exploring its siblings
- Siblings are explored in a left- to-right fashion.
- Depth-first search always expands the deepest node in the current frontier of the search tree
- Depth-first traversal: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6 \rightarrow 7$



# Depth First Search

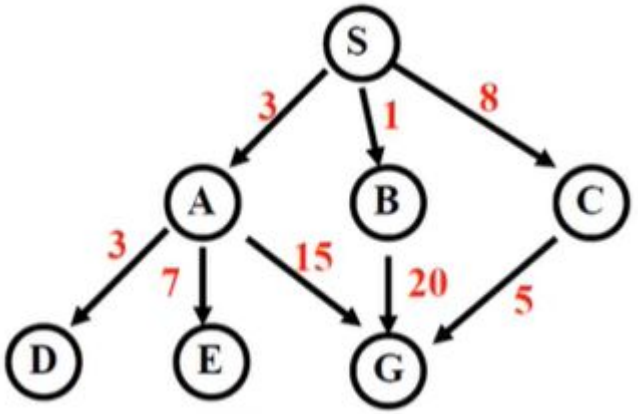
Apply Depth First search to find path from S to G



# Depth First Search

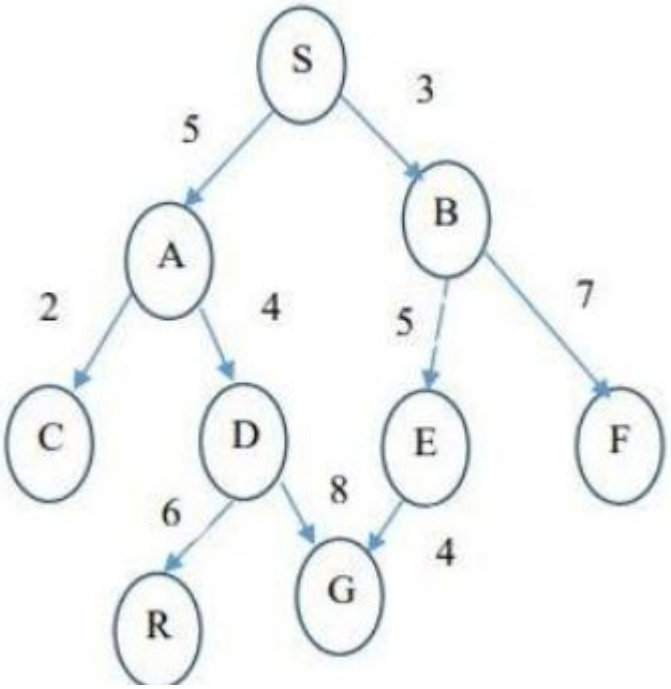
| Expanded node | Nodes list |
|---|---|
| | { S0 } |
| S0 | { A3 B1 C8 } |
| A3 | { D6 E10 G18 B1 C8 } |
| D6 | { E10 G18 B1 C8 } |
| E10 | { G18 B1 C8 } |
| G18 | { B1 C8 } |

- Solution path found is S A G, cost 18
- Number of nodes expanded (including goal node) = 5



# DFS

## Apply Depth First search to find path from S to G



# DFS Algorithm

The depth-first search algorithm is an instance of the graph-search algorithm; whereas breadth-first-search uses a FIFO queue, depth-first search uses a LIFO queue. A LIFO queue means that the most recently generated node is chosen for expansion. This must be the deepest unexpanded node because it is one deeper than its parent—which, in turn, was the deepest unexpanded node when it was selected. As an alternative to the GRAPH-SEARCH-style implementation, it is common to implement depth-first search with a recursive function that calls itself on each of its children in turn

```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

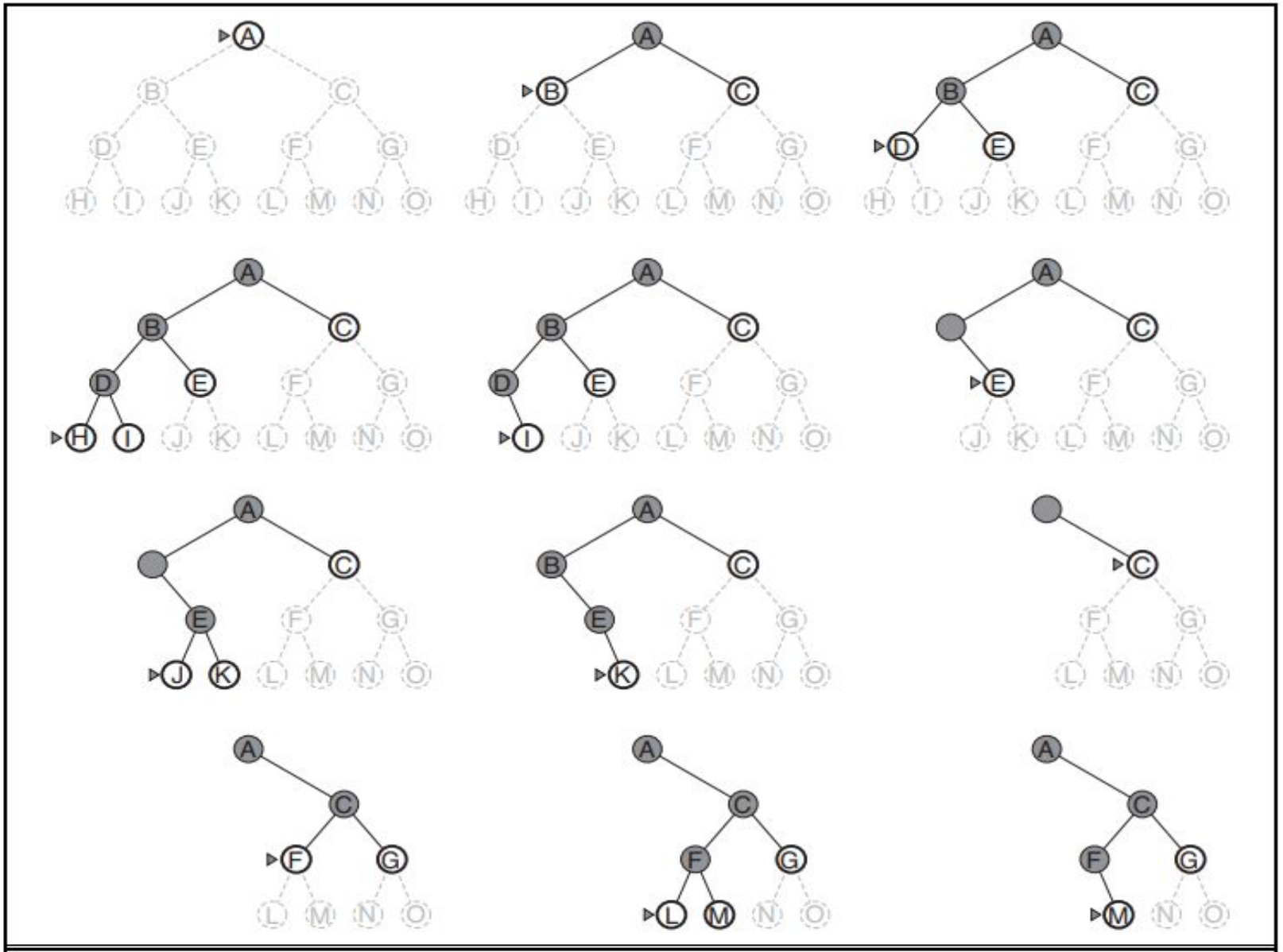


**Figure** Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and $M$ is the only goal node.

# Depth First Search - Complexity

Time Complexity:

- Let b be the maximal branching factor and m maximum depth of any node. Then the maximal number of nodes expanded is $1+b + b^2 + b^3 + \cdots + b^d = O(b^d)$

Space Complexity:

- Space taken is linear in the depth of the search tree
- For a state space with branching factor b and maximum depth d, depth-first search requires storage of only O(bd) nodes.
- O(bd)i.e. space linear in length of action sequence! Need only store a single path from the root to the leaf node, along with remaining unexpanded sibling nodes for each node on path.

Drawback of DFS: It can make a wrong choice and get stuck going down.

# DFS

Complete: Yes
Optimal: No

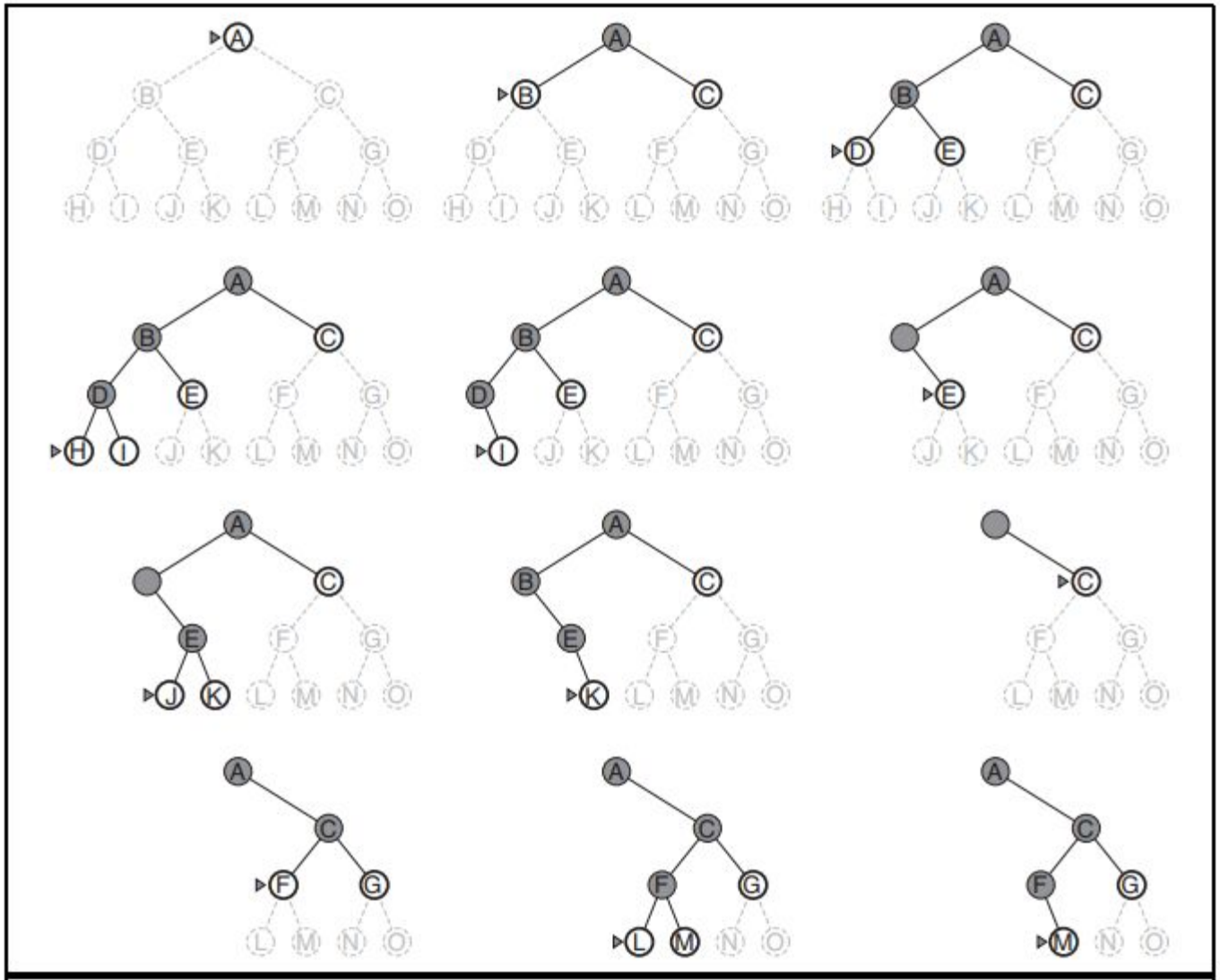


**Figure 3.16** Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and $M$ is the only goal node.

# Is DFS, Optimal ?

- Both versions (Graph Search and Tree Search) are nonoptimal. For example, in Figure 3.16, depthfirst search will explore the entire left subtree even if node C is a goal node. If node J were also a goal node, then depth-first search would return it as a solution instead of C, which would be a better solution; hence, depth-first search is not optimal.

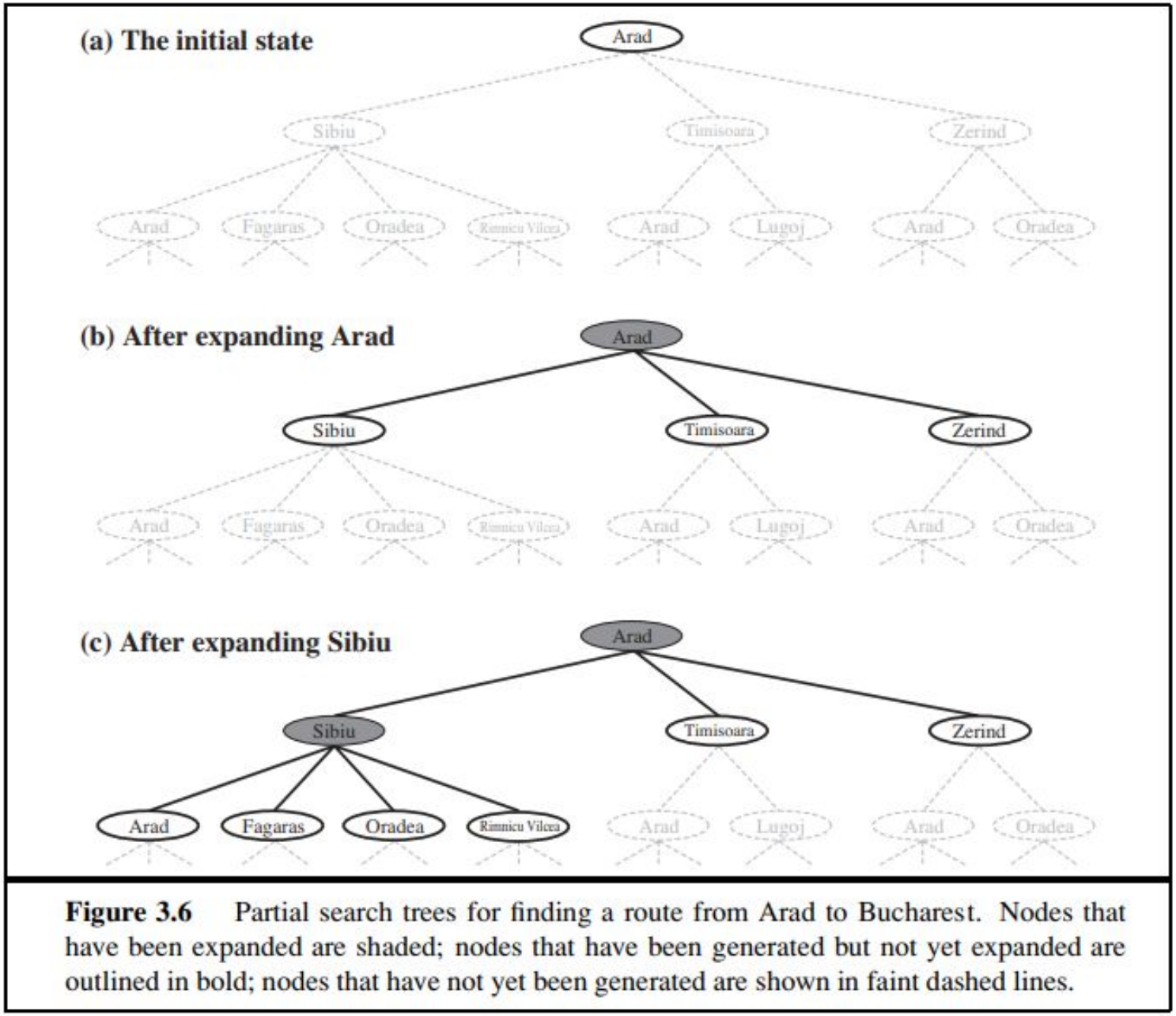
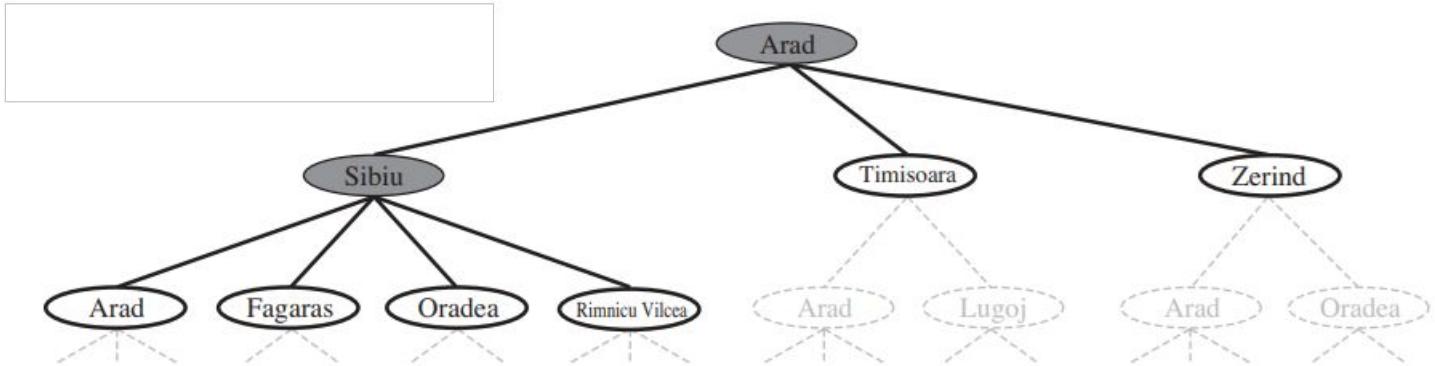



**Figure 3.6** Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

# Is DFS Complete ?

The properties of depth-first search depend strongly on whether the graph-search or tree-search version is used. The graph-search version, which avoids repeated states and redundant paths, is complete in finite state spaces because it will eventually expand every node. The tree-search version, on the other hand, is not complete—for example, in Figure 3.6 the algorithm will follow the Arad–Sibiu–Arad–Sibiu loop forever. Depth-first tree search can be modified at no extra memory cost so that it checks new states against those on the path from the root to the current node; this avoids infinite loops in finite state spaces but does not avoid the proliferation of redundant paths. In infinite state spaces, both versions fail if an infinite non-goal path is encountered



# Difference between DFS and BFS

DFS explores as deep as possible along one branch before backtracking, while BFS explores all neighbors at the current level before moving to the next level.

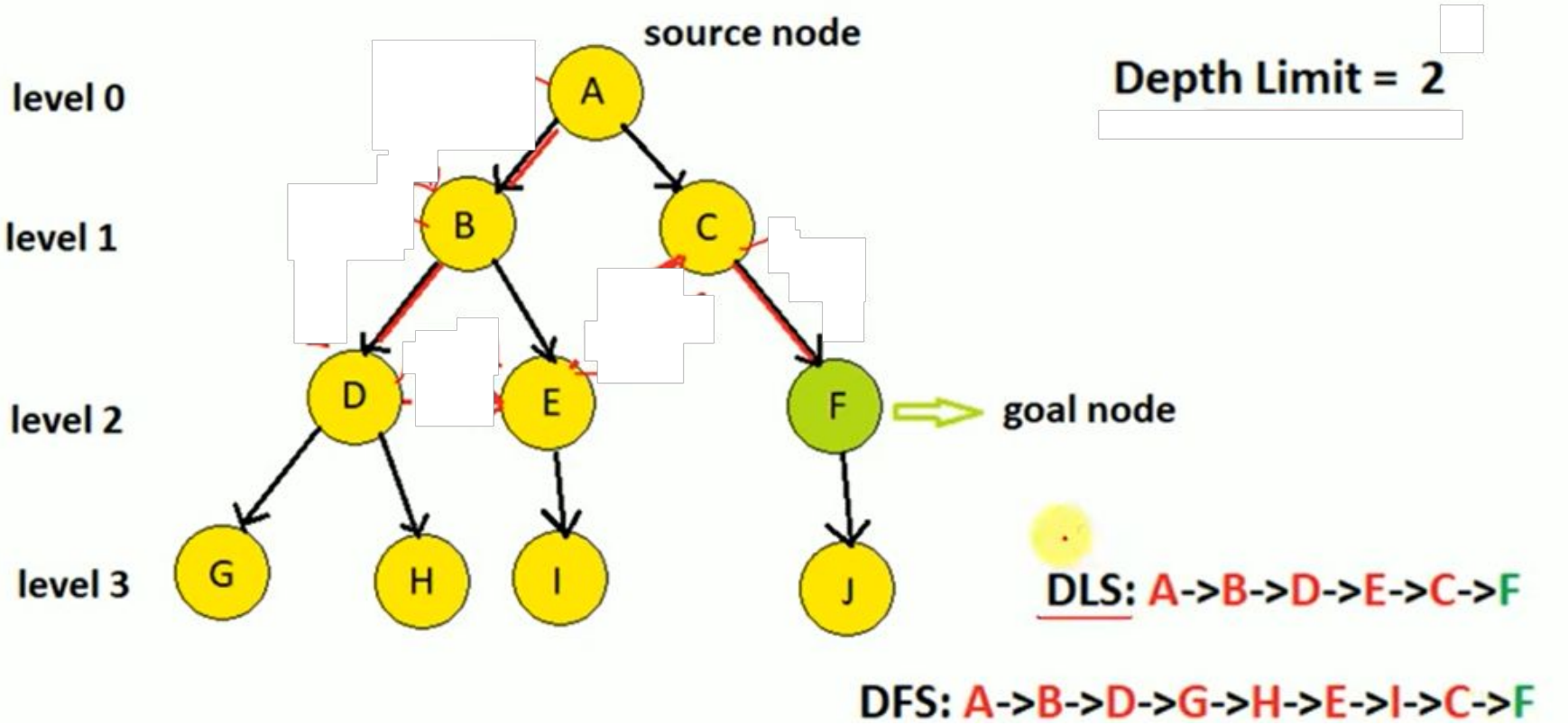
# Difference between DFS and BFS

DFS explores as deep as possible along one branch before backtracking, while BFS explores all neighbors at the current level before moving to the next level.

# Depth Limited Search (DLS)

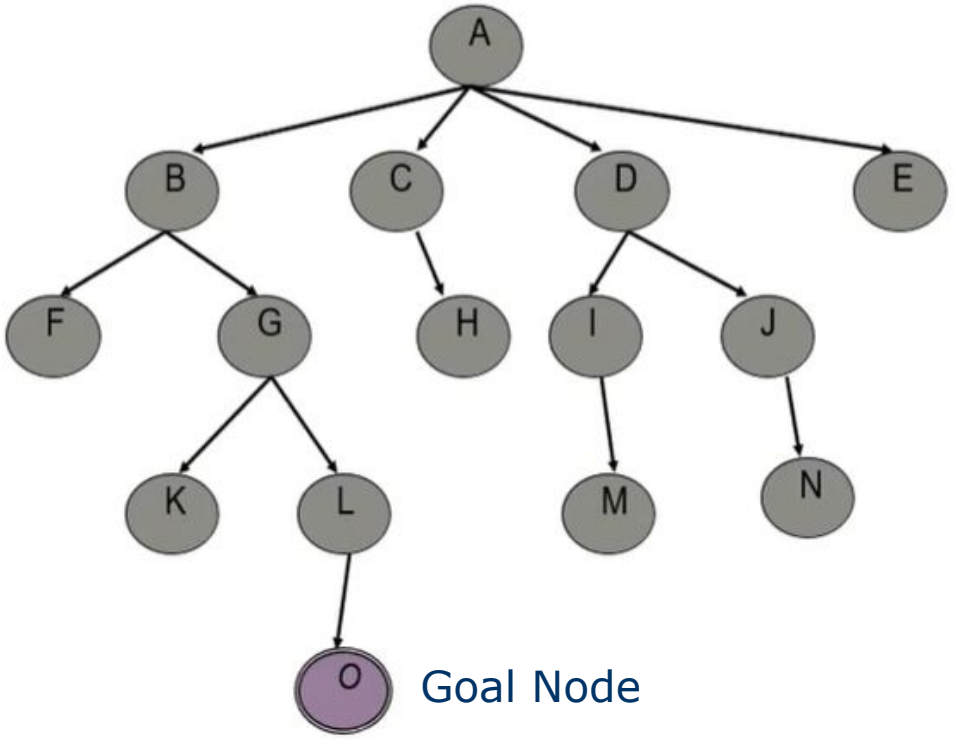
- The failure of Depth-First Search is the infinite state spaces, that can be overcome by supplying depth-first search with a predetermined depth limit.
- The depth limit 'l' i.e. the cut-off value or the maximum level of the depth to overcome the disadvantages of depth first search.
- Nodes at depth are treated as if they have no successors.
- This approach is called Depth-Limited Search.
- The depth limit solves the infinite-path problem.
- Notice that depth-limited search can terminate with two kinds of failure:
  - the standard failure value indicates no solution;
  - the cutoff value indicates no solution within the depth limit.

# DFS and DLS


source node
level 0
level 1
level 2
level 3
A
B
C
D
E
F
G
H
I
J
Depth Limit = 2
goal node
DLS: A->B->D->E->C->F
DFS: A->B->D->G->H->E->I->C->F

# Depth Limited Search

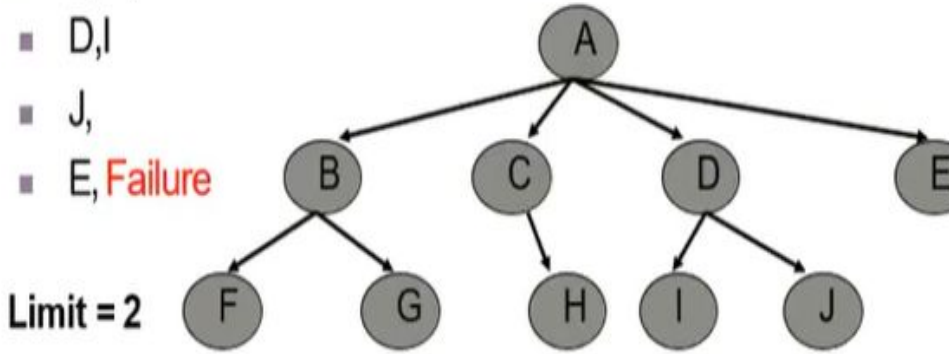
- Limit 0
- Limit 1
- Limit 2

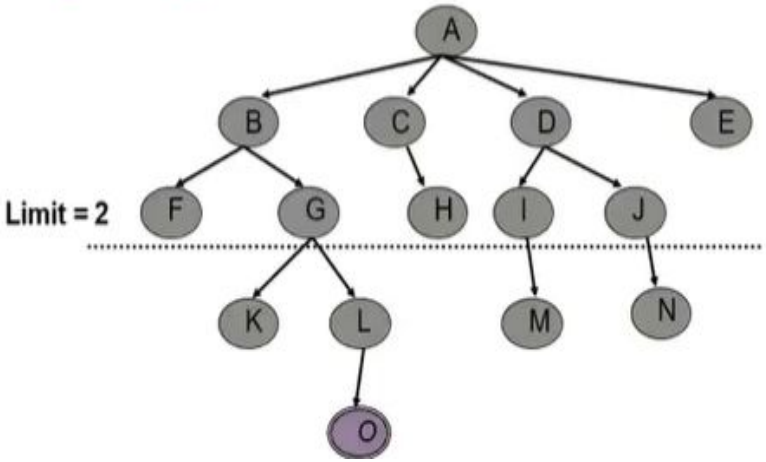


# Depth Limited Search

- DLS algorithm returns Failure (no solution)
- The reason is that the goal is beyond the limit (Limit =2): the goal depth is (d=4)

- A,B,F,
- G,
- C,H,
- D,I
- J,
- E, Failure



# Depth Limited Search

Let d be the depth of shallowest goal node

**Incompleteness** if l < d

**Non optimal** if l < d, because not guaranteed to find the shortest solution in the search technique.

Depth-first search can be viewed as a special case of depth-limited search with l =∞

l is the Limit.

# Depth Limited Search

Time Complexity: $O(b^l)$
Space Complexity: O(bl)

b: branching factor
l: limit

Space Complexity:O(bl) i.e. space linear in length of action sequence! Need only store a single path from the root to the leaf node, along with remaining unexpanded sibling nodes for each node on path.

# Depth Limited Search

= depth first search with depth limit of $l$

i.e nodes at level $l$ have no successors

| Depth limited search (limit) |
| --- |
| Let fringe be a list containing the initial state<br>Loop<br>if fringe is empty return failure<br>Node ← remove-first (fringe)<br>if Node is a goal<br>then return the path from initial state to Node<br>else if depth of Node = limit return cutoff<br>else add generated nodes to the front of fringe<br>End Loop |

# Recursive implementation of depth-limited tree search.

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

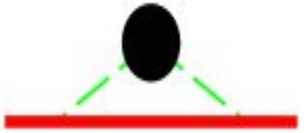
# Iterative Deepening Search

- The Iterative Deepening Depth First Search is simply called as iterative deepening search (IDS)
- Combine the benefits of depth-first and breadth-first search to finds the best solution.
- It gradually increase the limit from 0,1,2 and so on until reaches the goal.
- It will terminate when the depth limit reaches d, depth of the shallowest goal node, with success message.

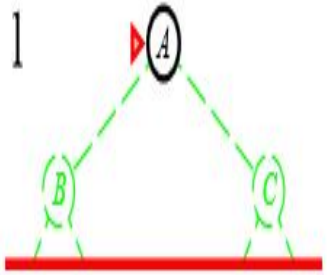
# Iterative deepening depth first search

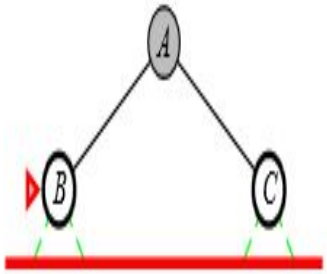
Limit = 0

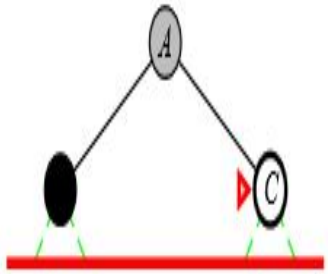


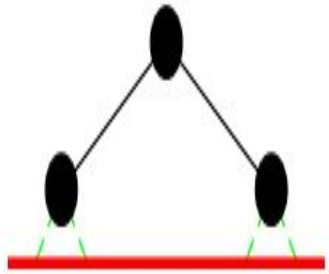



Limit = 1

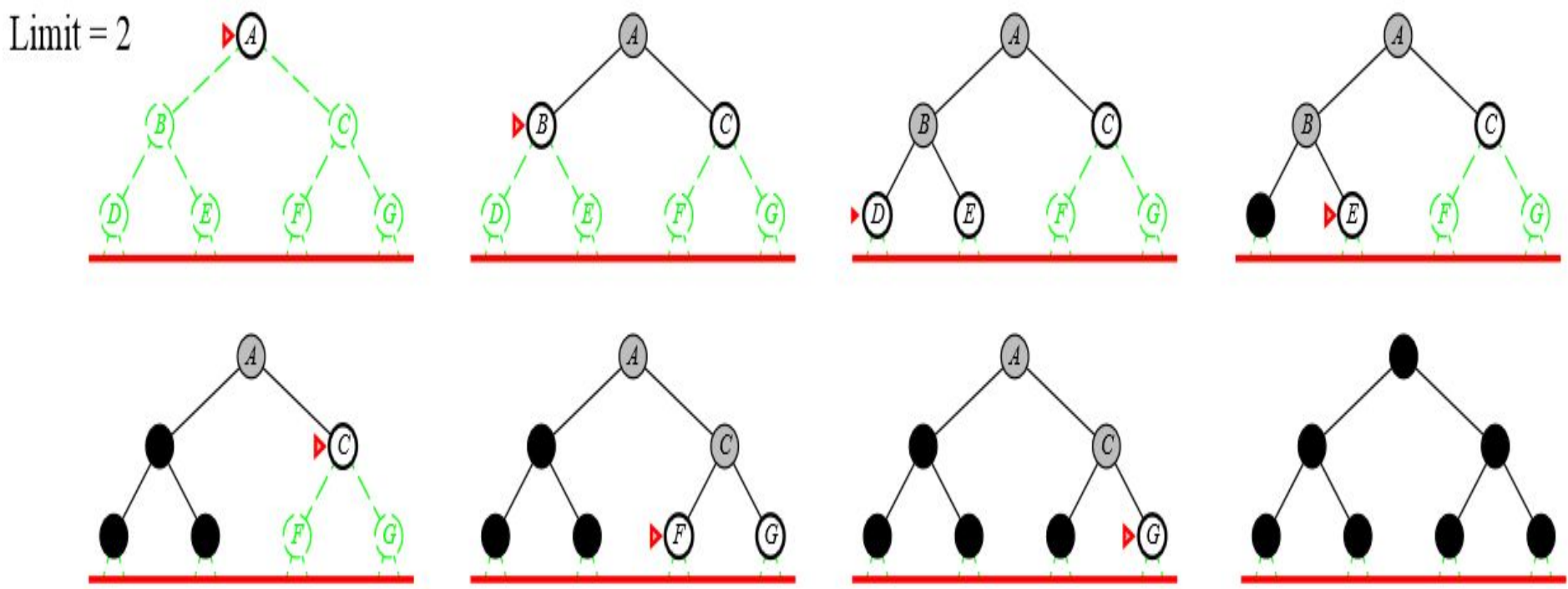


# Iterative deepening depth first search



# Iterative deepening depth first search

- May seem wasteful because states are generated multiple times
- but actually not very costly, because nodes at the bottom level are generated only once
- The overhead of these multiple expansions is small, because most of the nodes are towards leaves (bottom) of the search tree:
  - thus, the nodes that are evaluated several times (towards top of tree) are in relatively small number.
- Iterative depending is the preferred uninformed search method when the search space is large and the depth of the solution is unknown

# Iterative deepening depth first search

- This method finds best depth limit

```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH( problem, depth)
        if result ≠ cutoff then return result
    end
```

# Iterative deepening depth first search (IDDFS)

The time complexity of IDDFS in a (well-balanced) tree works out to be the same as breadth-first search, i.e.
$O(b^d)$, where $b$ is the branching factor and $d$ is the depth of the goal.

In an iterative deepening search, the nodes at depth $d$ are expanded once, those at depth $d-1$ are expanded twice, and so on up to the root of the search tree, which is expanded $d+1$ times
So the total number of expansions in an iterative deepening search is

$$b^d + 2b^{d-1} + 3b^{d-2} + \cdots + (d-1)b^2 + db + (d+1) = \sum_{i=0}^{d} (d+1-i)b^i$$

For $b = 10$ and $d=5$ the number is

$$\sum_{i=0}^{5} (5+1-i)10^i = 6 + 50 + 400 + 3000 + 20000 + 100000 = 123456$$

# Iterative deepening depth first search (IDDFS)

For b = 10 and d=5 the number is

$$b^d + 2b^{d-1} + 3b^{d-2} + \cdots + (d-1)b^2 + db + (d+1) = \sum_{i=0}^{d}(d+1-i)b^i$$

$$\sum_{i=0}^{5}(5+1-i)10^i = 6 + 50 + 400 + 3000 + 20000 + 100000 = 123456$$

Number of Nodes Generated for Breadth First Search is
= 10 + 100 + 1, 000 + 10, 000 + 100, 000 = 111, 110

For b = 10 , IDDFS expands only 11% more than the number of nodes expanded by breadth-first-search.

In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known.

# Comparison between IDDFS and BFS

Iterative deepening with Depth-First Search uses much less memory than Breadth-First Search.

At any point in the DFS, the amount of memory in use proportional to the neighbors of a single path through the search tree. (Possibly less if neighbors can be generated in an ordered fashion.) BFS uses an amount of memory proportional to the entire width of the search tree. BFS quickly becomes infeasible.

The tradeoff is that iterative deepening revisits the same states many times. But, if the number of states is growing exponentially, then the amount of time spent re-visiting earlier states adds only a constant factor.

# Iterative deepening depth first search

- Time Complexity

$db^1 + (d-1)b^2 + \ldots\ldots\ldots + 2b^{d-1} + 1b^d = O(b^d)$

- Space Complexity O(bd)

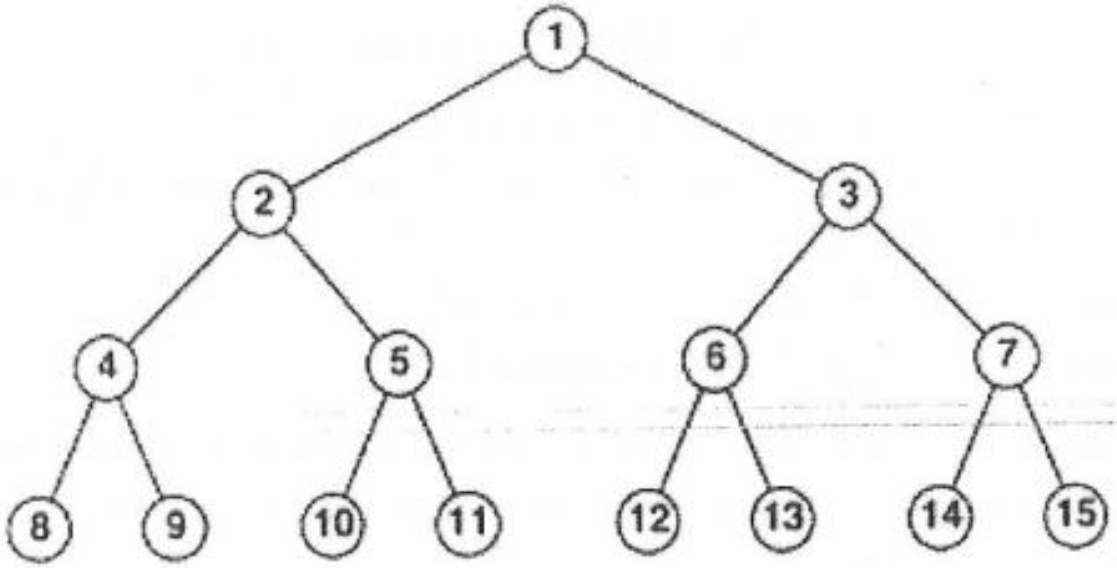
# Iterative deepening depth first search

Completeness: Yes

Optimality: Yes, if step cost=1

# Question

(b) Consider a state space given below.




(i) If the goal state is numbered '11', list the order in which the states will be visited using (1) breadth first search (2) depth first search (3) depth limited search with a limit 2 (depth of the root is 0) (4) iterative deepening search.

(ii) If iterative deepening search is used to traverse the state space, how many times will the state space tree be constructed to reach the goal?

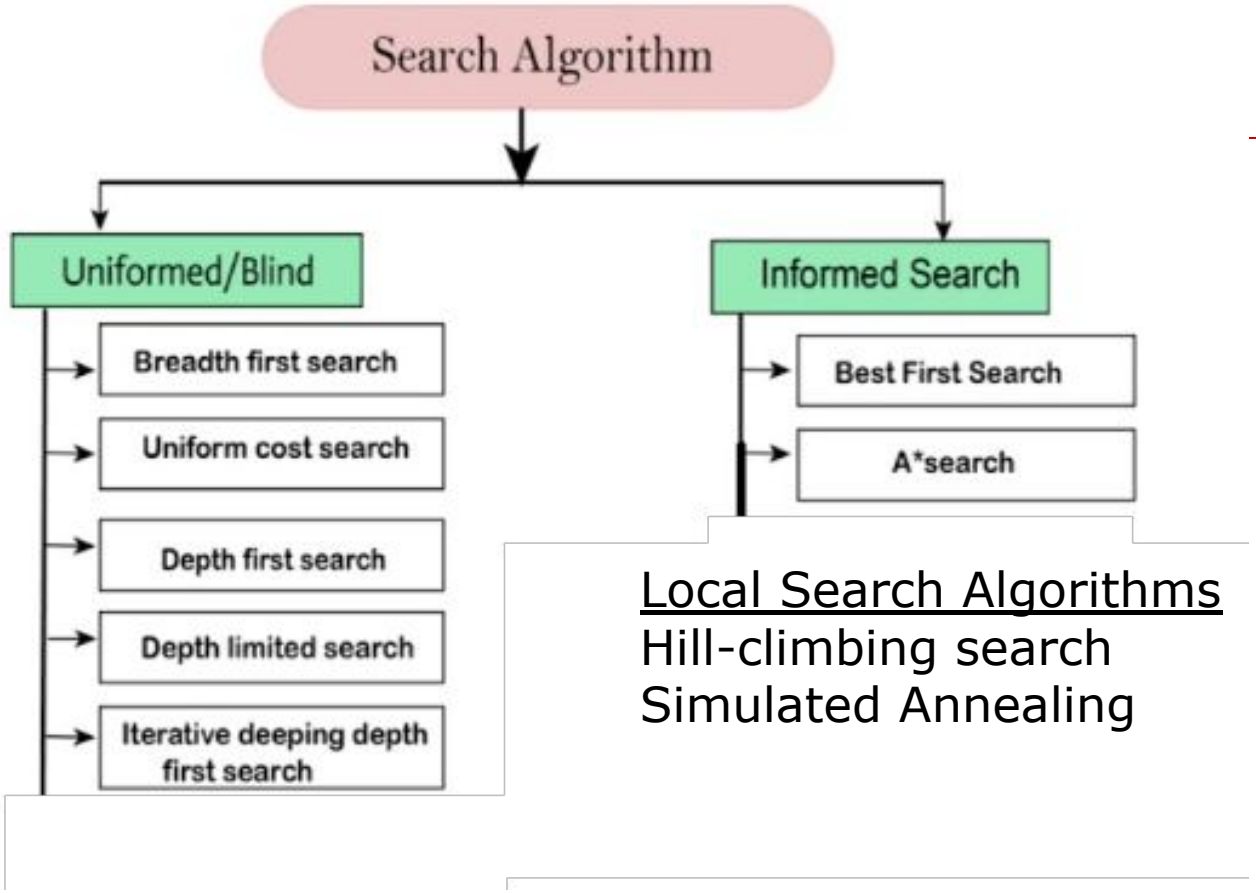
# Answer

BFS: 1,2,3,4,5,6,7,8,9,10,11
DFS: 1, 2, 4, 8, 9, 5, 10, 11
DLS (depth limit 2): 1, 2, 4, 5, 3, 6, 7

IDS:
First iteration: 1
Second iteration: 1, 2, 3
Third iteration: 1, 2, 4, 5, 3, 6, 7
Fourth iteration: 1, 2, 4, 8, 9, 5, 10, 11

# Summary of uninformed search algorithms

| Criterion | Breadth First | Uniform Cost | Depth First | Depth Limited | Iterative Deepening |
|---|---|---|---|---|---|
| Complete ? | Yes | Yes | No | No | Yes |
| Time | $O(b^{d+1})$ | $O(b^{C/\epsilon})$ | $O(b^{d})$ | $O(b^{l})$ | $O(b^{d})$ |
| Space | $O(b^{d+1})$ | $O(b^{C/\epsilon})$ | $O(b^{d})$ | $O(b^{l})$ | $O(b^{d})$ |
| Optimal ? | Yes | Yes | No | No | Yes |




Local Search Algorithms
Hill-climbing search
Simulated Annealing

# Uninformed Search Strategies

- Uninformed (blind, brute-force) search algorithm generates the search tree without using any domain specific knowledge.
- No additional information about states beyond that provided in the problem definition.
- These algorithms generate successors using the successor function
- Knlowedge about goal state (knowledge about a goal state and a non-goal states)
- No information about the **path cost** from the current state to goal state.

| Uninformed search | Informed search |
| --- | --- |
| ➢ look for solutions by **systematically** generating new states and checking each of them against the goal.<br><br>1. It is very **inefficient** in most cases.<br>2. Most successor states are "obviously" a bad choice.<br>3. Such strategies do not use problem-specific knowledge | 1. They are almost always **more efficient** than uninformed strategies.<br>2. May reduce time and space complexities.<br>3. Evaluation function f(n) measures distance to the goal.<br>4. Order nodes in **Frontier** according to f(n) and decide which node to expand next. |

# Meaning of the Word

## HEURISTIC:

- To discover
- Involving or serving as an aid to learning, discovery, or problem-solving by experimental and especially trial-and-error methods

# Informed Search or Heuristic Search

- Informed search methods use problem specific knowledge and are more efficient
- Heuristic means "rule of thumb"
- "Heuristics are criteria, methods or principles for deciding which among several alternative courses of action promises to be the most effective in order to achieve some goal"
- In informed search heuristics are used to identify most promising search path.

# Heuristic Function

## A heuristic function, denoted h(n):

HEURISTIC FUNCTION h(n) = estimated cost of the cheapest path from the state at node n to a goal state

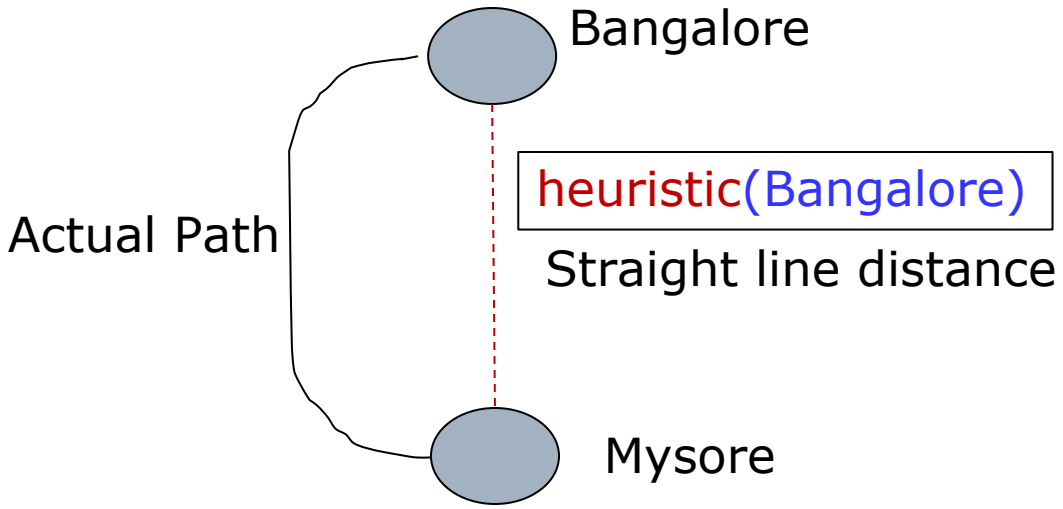
h(n) is arbitrary, nonnegative, problem-specific functions.

If n is a goal node, then h(n)=0.

# Example of heuristic function

1. Want path from Bangalore to Mysore

heuristic(Bangalore) = EuclideanDistance(Banglore, Mysore)



# Example of heuristic function

## 2. 8-puzzle

## Heuristic: Number tiles out of place

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| | 7 | 5 |

Initial State n

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Goal State

heuristic(n) = 5

# Example of heuristic function

## 2. 8-puzzle

## Heuristic: Manhattan Distance

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| | 7 | 5 |

Initial State n

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Goal State

heuristic(n) = 1 + 1 + 0 + 0 + 0 + 1 + 1 +2 = 6

Manhattan Distance Heuristic: Another heuristic for 8-puzzle is the Manhattan distance heuristic. This heuristic sums the distance that the tiles are out of place. The distance of a tile is measured by the sum of the differences in the x-positions and the y-positions.

# Heuristic Function

- A heuristic function at a node n is an estimate of the optimum cost from the current node to a goal.
- It is denoted by *h(n)*.
- *h(n)* = Estimated cost of the cheapest path from node n to a goal node
- *h(goal)* = 0

# Greedy Best-first search

- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms.
- It uses the heuristic function and search.
- With the help of best-first search, at each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the minimum cost is estimated by heuristic function

# Greedy Best-first search

- The evaluation function is f(n) = h(n)
- Were, h(n)= estimated cost from node n to the goal.

- Greedy search ignores the cost of the path that has already been traversed to reach n
- Therefore, the solution given is not necessarily optimal

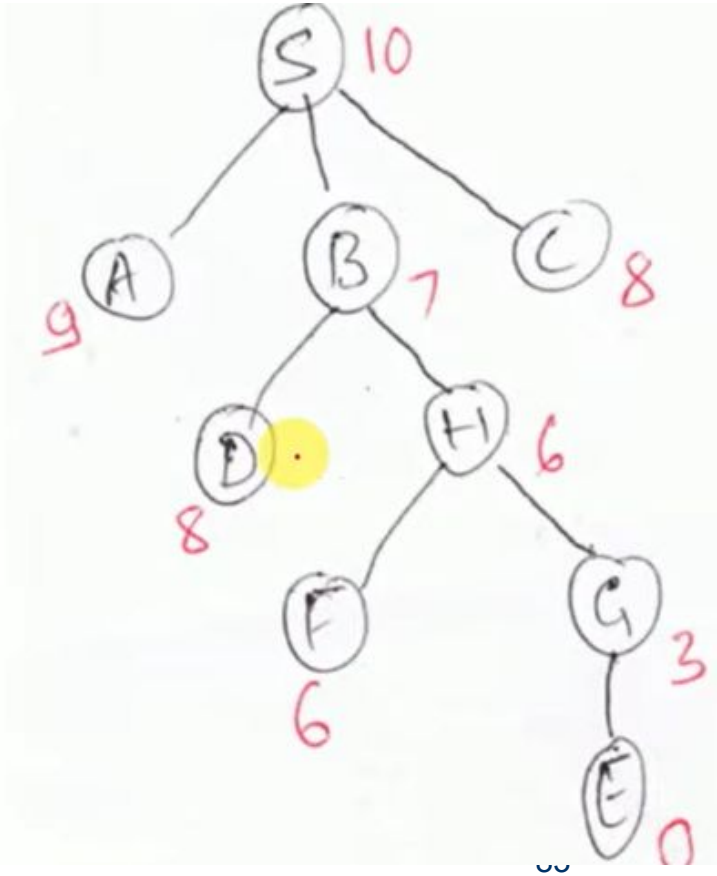
# Greedy Best First Search Algorithm

1. Create 2 empty lists: OPEN and CLOSED
2. Start from the initial node (say N) and put it in the 'ordered' OPEN list
3. Repeat the next steps until the GOAL node is reached
    1. If the OPEN list is empty, then EXIT the loop returning 'False'
    2. Select the first/top node (say N) in the OPEN list and move it to the CLOSED list. Also, capture the information of the parent node
    3. If N is a GOAL node, then move the node to the Closed list and exit the loop returning 'True'. The solution can be found by backtracking the path
    4. If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list
    5. Reorder the nodes in the OPEN list in ascending order according to an evaluation function f(n).

# Problem

Apply the Greedy best first search to find the solution path from S to G.

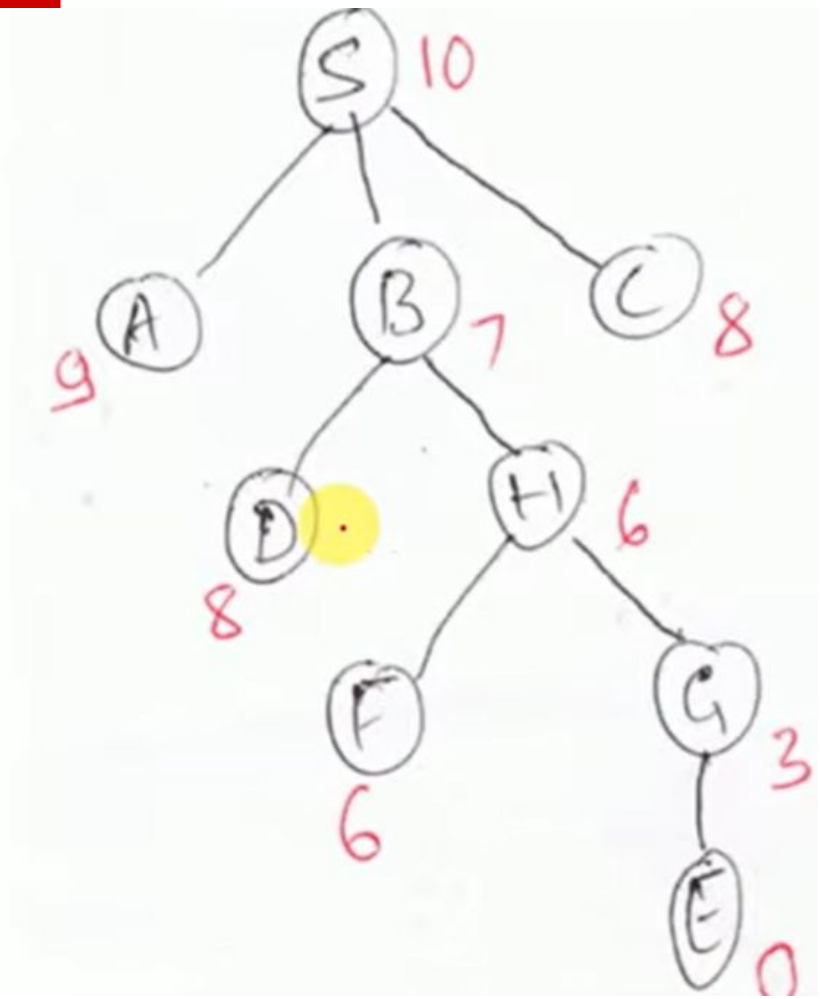
| Open | | | Closed | |
|---|---|---|---|---|
| Node | H(n) | | Node | Parent |
| S | 10 | | | |
| | | | | |



# Solution

- Final Iteration

| Open | | | Closed | |
|---|---|---|---|---|
| Node | H(n) | | Node | Parent |
| F | 6 | | S | |
| C | 8 | | B | S |
| D | 8 | | H | B |
| A | 9 | | G | H |
| | | | E | G |

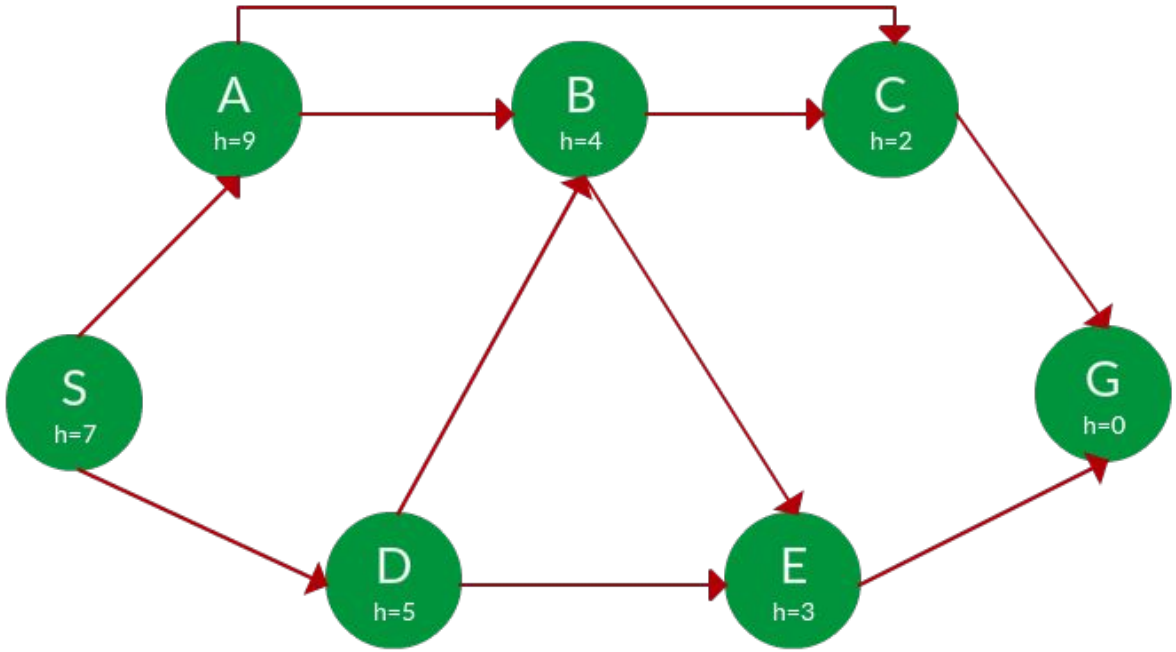



Path

S → B → H → G → E

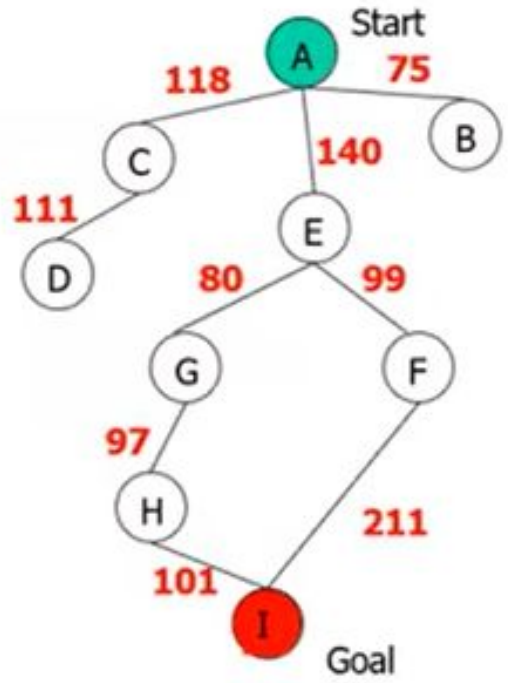
# Problem

Apply the Greedy best first search to find the solution path from S to G. Write all steps as well as open and closed lists for full marks. S(h=7),A(h=9),B(h=4),C(h=2),D(h=5), E(h=3),G(h=0)



# Problem

Apply the Greedy best first search to find the solution path from A to I.




| State | Heuristic: h(n) |
|---|---|
| A | 366 |
| B | 374 |
| C | 329 |
| D | 244 |
| E | 253 |
| F | 178 |
| G | 193 |
| H | 98 |
| I | 0 |

$f(n) = h(n)$ = straight-line distance heuristic

# Greedy Search

- Idea - Expand the node with the smallest estimated cost to reach the goal
- use a heuristic function $f(n) = h(n)$
- $h(n)$ estimates the distance remaining to a goal.
  - Not Optimal
  - In complete

# Disadvantages

- Greedy best-first search can start down an infinite path and never return to try other possibilities, it is incomplete
- Because of its greediness the search makes choices that can lead to a dead end; then one backs up in the search tree to the deepest unexpanded node
- Greedy best-first search resembles depth-first search in the way it prefers to follow a single path all the way to the goal, but will back up when it hits a dead end
- The quality of the heuristic function determines the practical usability of greedy search

# A* Search

- Greedy Best First Search minimizes a heuristic h(n) which is an estimated cost from a current state n to the goal state.
- Greedy Best First Search is efficient but it is not optimal and not complete.
- Uniform Cost Search minimizes the cost g(n) from the initial state to current state n.
- Uniform Cost Search is optimal and complete but not efficient.
- **A* Search**: Combine Greedy Best First Search and Uniform Cost Search to get an efficient algorithm which is complete and optimal.

# A* Search

- **A* search** evaluates nodes by combining g(n), the cost to reach the node and h(n), the cost to get from the node to the goal.
- **f(n) = g(n) + h(n)**
- f(n) is the evaluation function which gives the cheapest solution cost
- g(n) is the exact cost to reach node n from the initial state.
- h(n) is an estimation of the assumed cost from current state (n) to reach the goal.

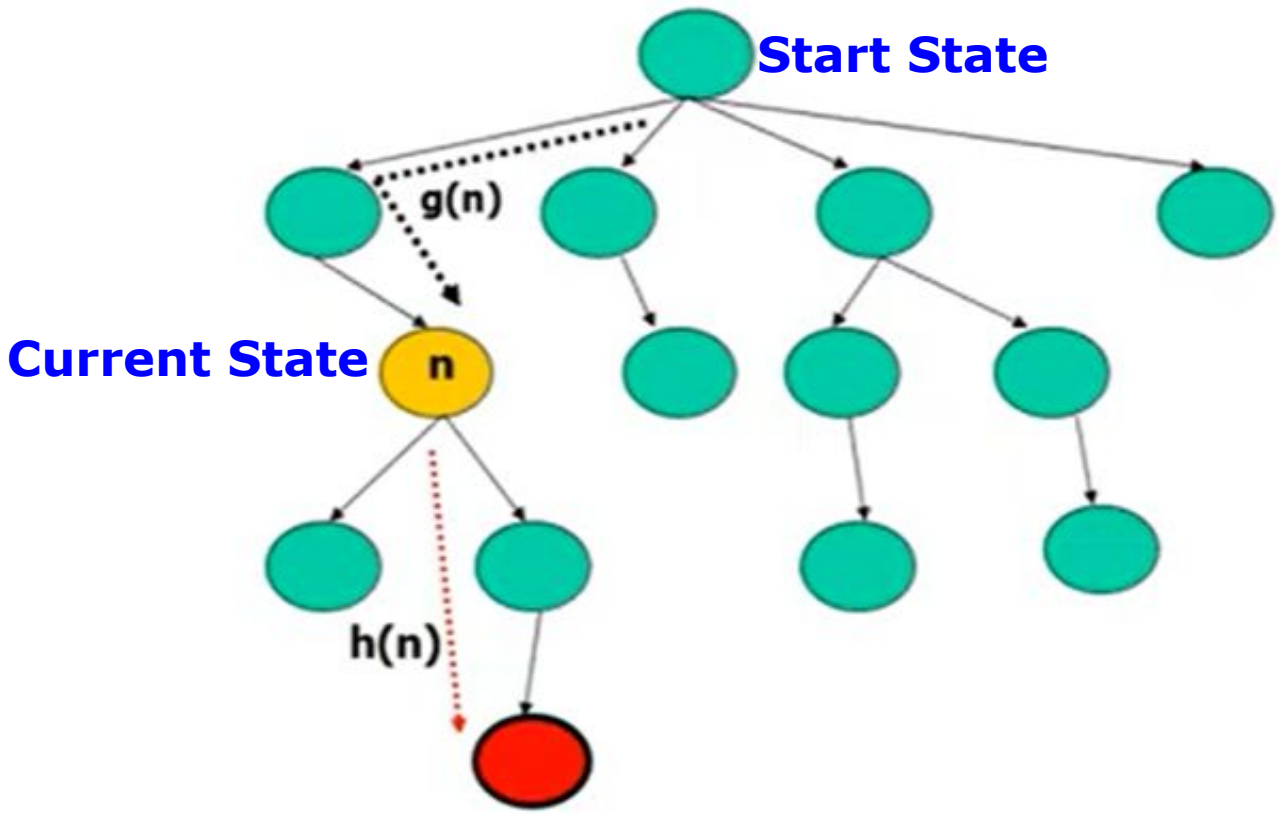
# A* Search Algorithm

A* is a best first search

- $f(n) = g(n) + h(n)$
  - where $g(n)$ = sum of edge costs from start to n
  - $h(n)$ = estimate of lowest cost path from n to goal

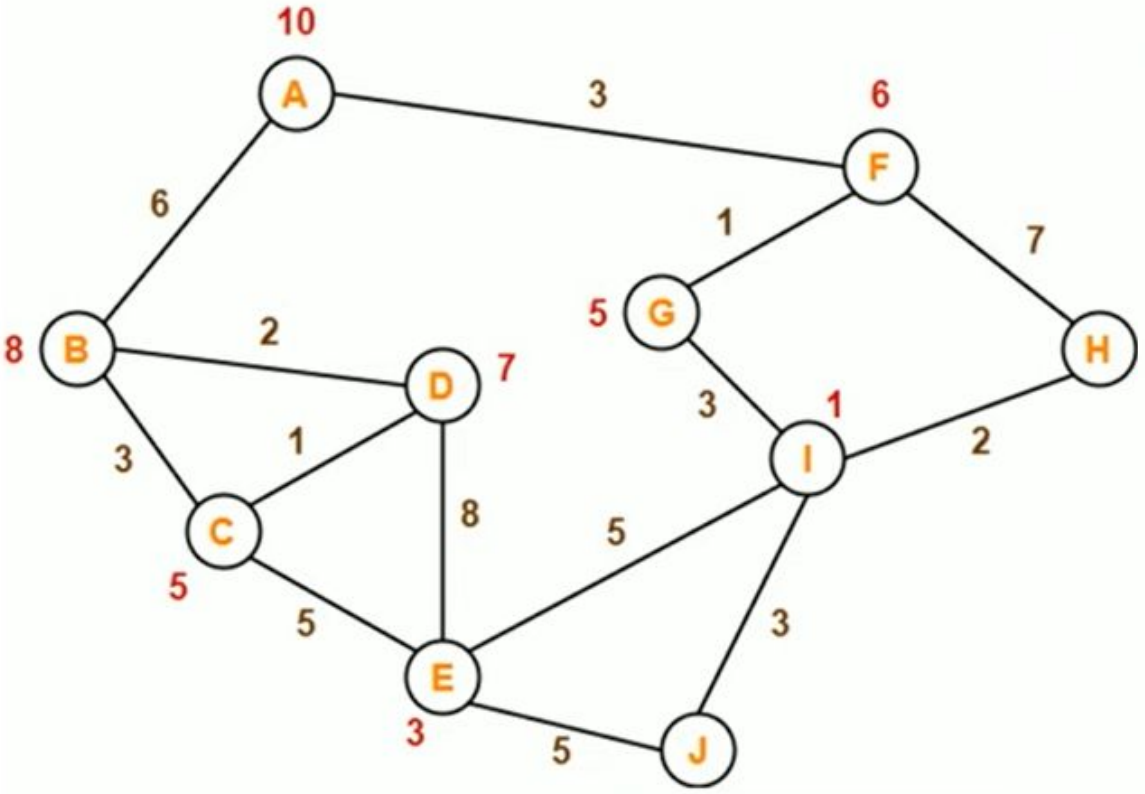
- $f(n)$ = actual distance so far + estimated distance remaining

# A* Search



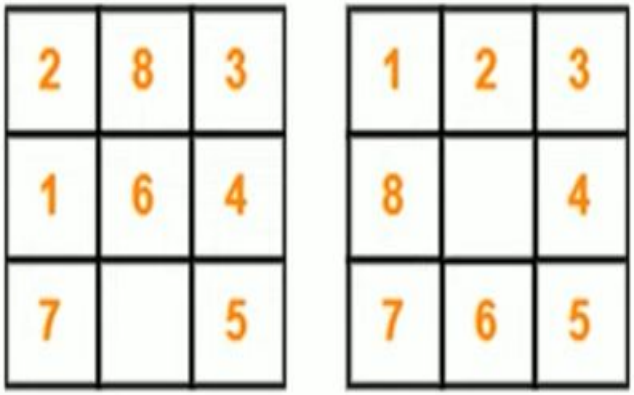

$$f(n) = g(n)+h(n)$$

# Problem: Find most cost effective path to reach from start state A to final state J using A * search




- The numbers written on edges represent the distance between the nodes.
- The numbers written on nodes represent the heuristic value.

Given an initial state of a 8-puzzle problem and final state to be reached-

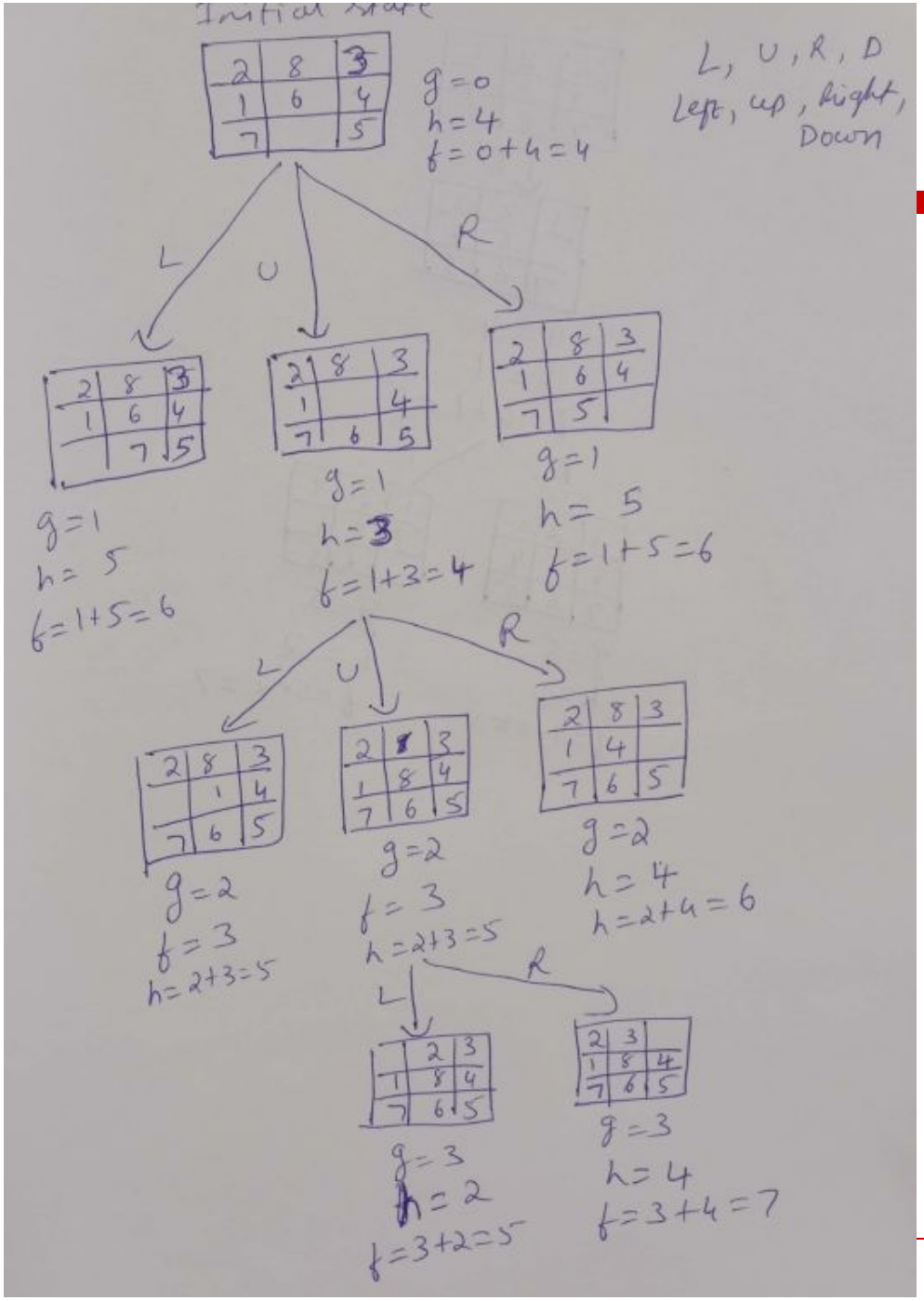


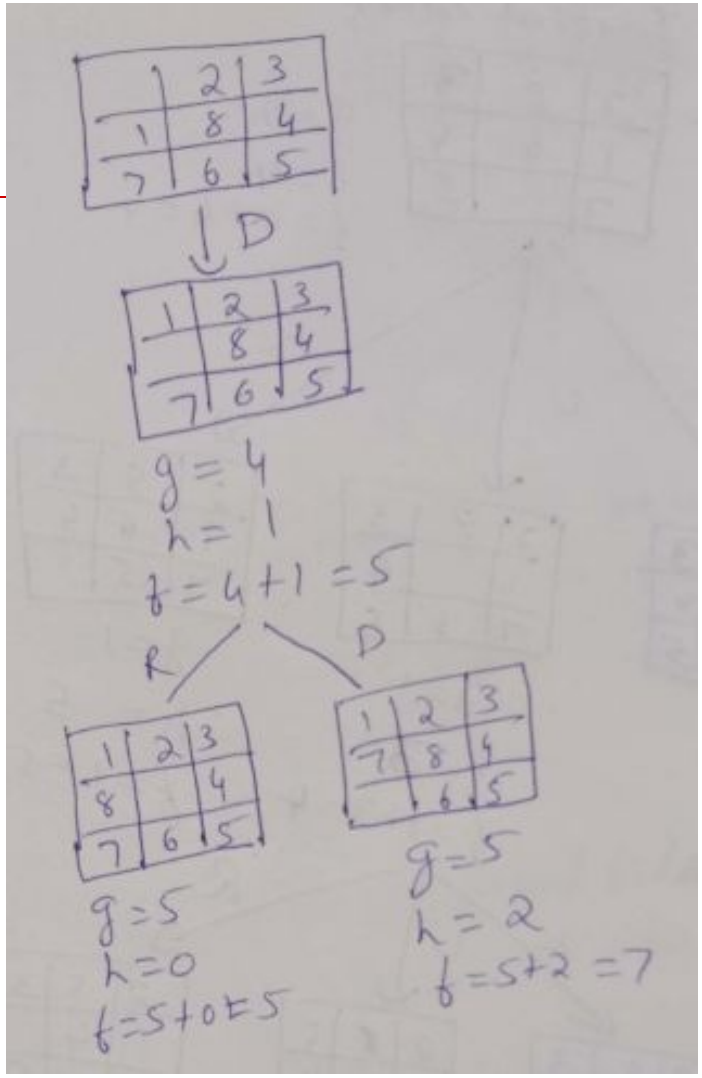

Initial State Final State

- Find the most cost-effective path to reach the final state from initial state using A* Algorithm.

$$f(n) = g(n) + h(n)$$

- Consider **g(n) = Depth of node** and **h(n) = Number of misplaced tiles.**

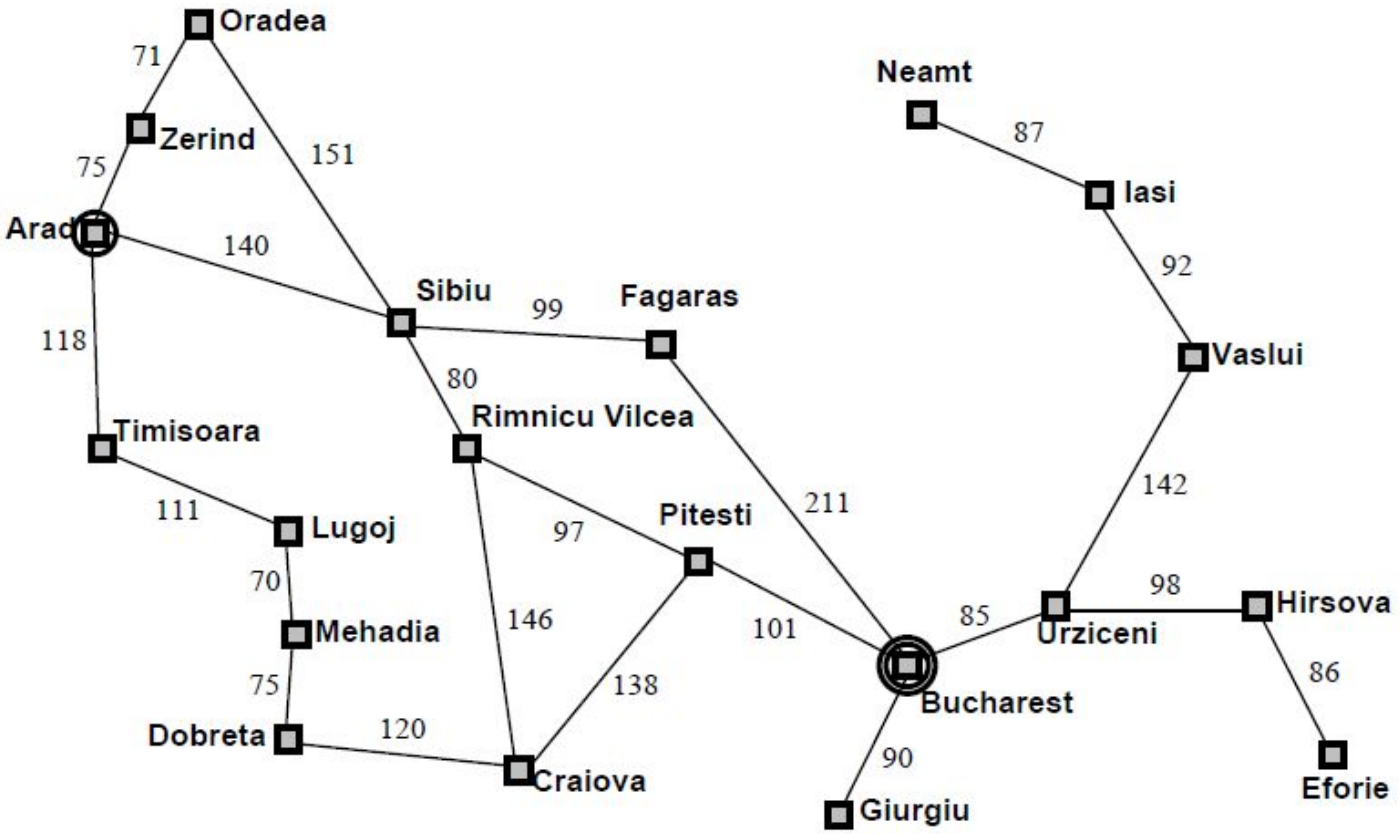

Initial state
L, U, R, D
Left, up, Right, Down
g=0
h=4
f=0+4=4
L
U
R
g=1
h=5
f=1+5=6
g=1
h=3
f=1+3=4
g=1
h=5
f=1+5=6
g=2
f=3
h=2+3=5
g=2
h=4
f=2+4=6
g=2
f=3
h=2+3=5
g=3
h=2
f=3+2=5
g=3
h=4
f=3+4=7


D
g=4
h=1
f=4+1=5
R
D
g=5
h=0
f=5+0=5
g=5
h=2
f=5+2=7

# The A* Algorithm

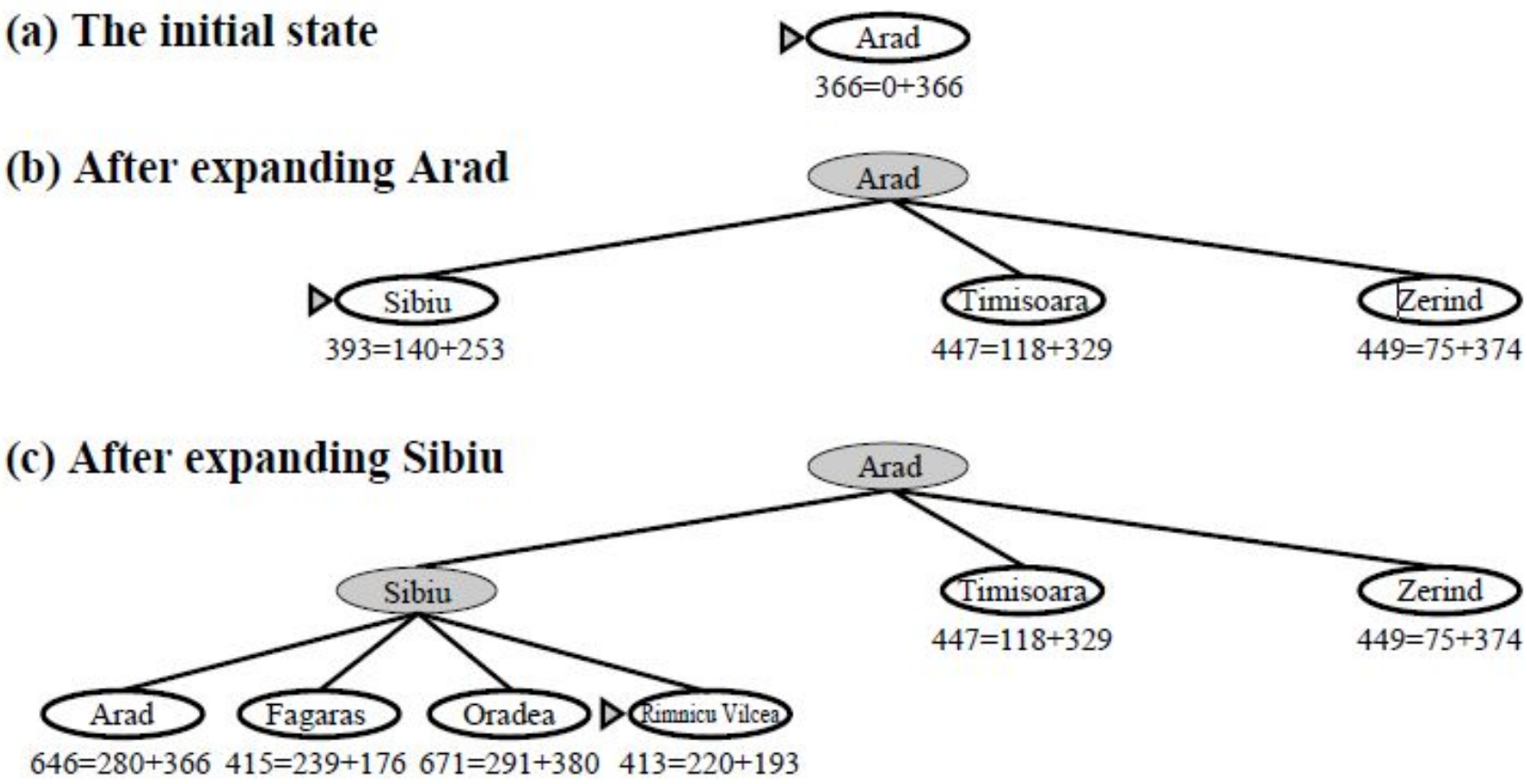
**function** A* Search(*problem*) **returns** a solution, or failure
  *node* ← a node $n$ with $n$.*state*=*problem.initialState*, $n.g = 0$
  *frontier* ← a priority queue ordered by ascending $g + h$, only element $n$
  **loop do**
    **if** *empty?(frontier)* **then return** failure
    $n$ ← *pop(frontier)*
    **if** *problem.goalTest*($n$.*state*) **then return** *solution*($n$)
    **for each** *action* $a$ **in** *problem.actions*($n$.*state*) **do**
      $n'$ ← *childNode*(*problem*,$n$,$a$)
      *insert*($n'$, $g(n') + h(n')$, *frontier*)

# A* Example

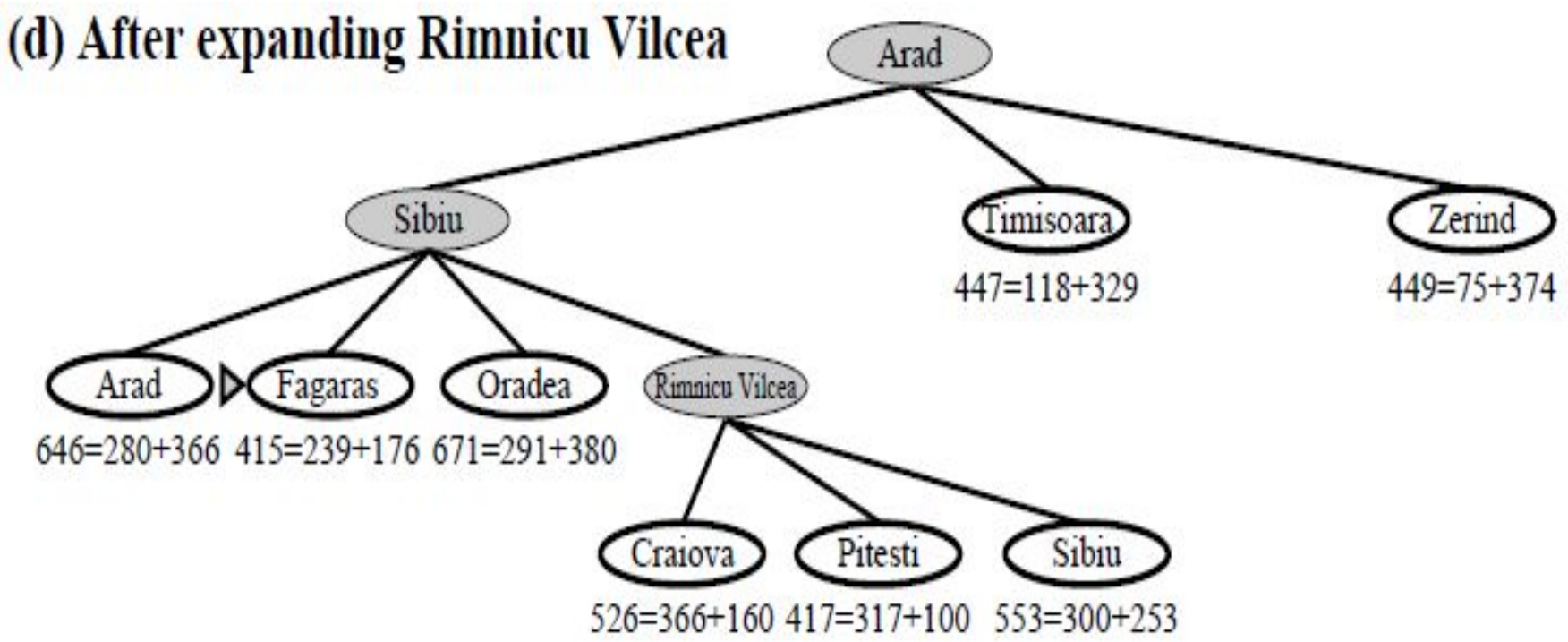



| | |
|---|---|
| Arad | 366 |
| Bucharest | 0 |
| Craiova | 160 |
| Drobeta | 242 |
| Eforie | 161 |
| Fagaras | 176 |
| Giurgiu | 77 |
| Hirsova | 151 |
| Iasi | 226 |
| Lugoj | 244 |
| Mehadia | 241 |
| Neamt | 234 |
| Oradea | 380 |
| Pitesti | 100 |
| Rimnicu Vilcea | 193 |
| Sibiu | 253 |
| Timisoara | 329 |
| Urziceni | 80 |
| Vaslui | 199 |
| Zerind | 374 |

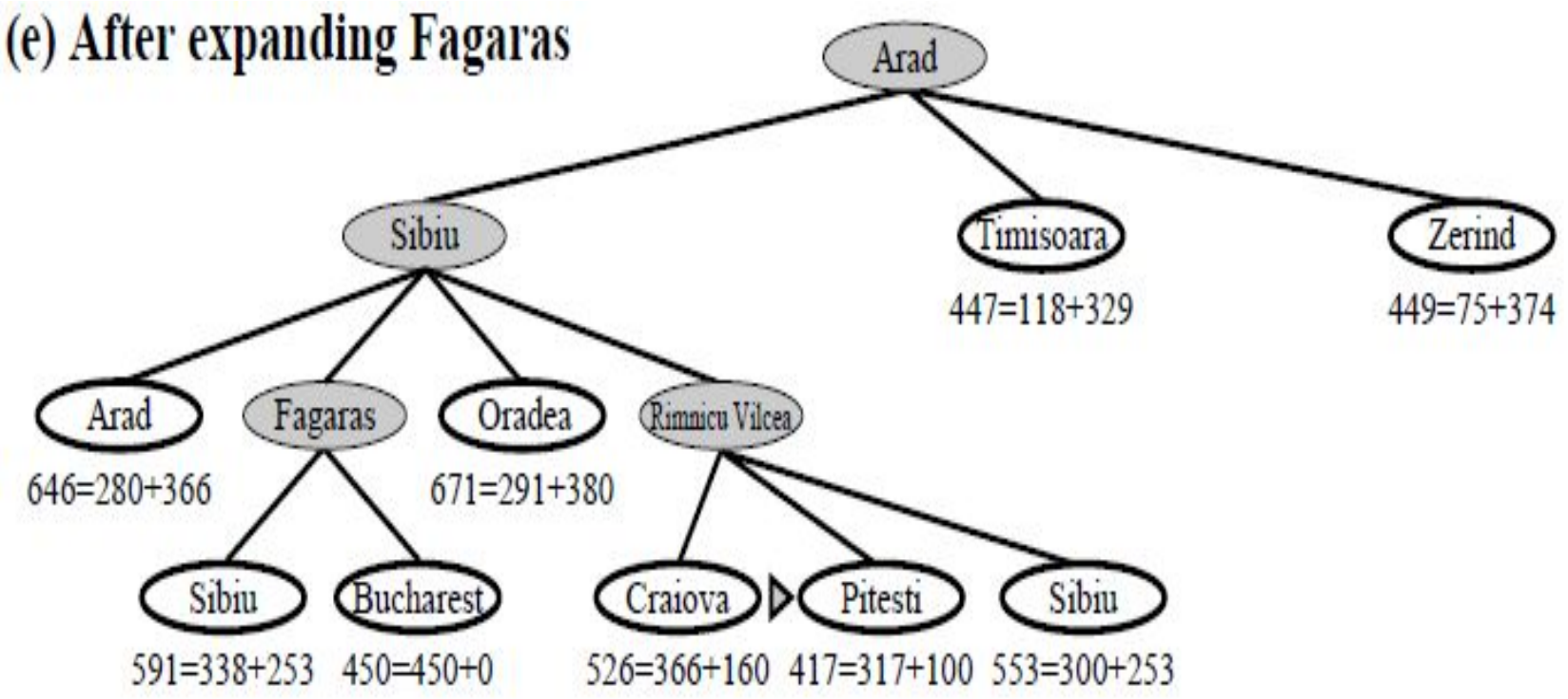
# A* Example - From Arad to Bucharest



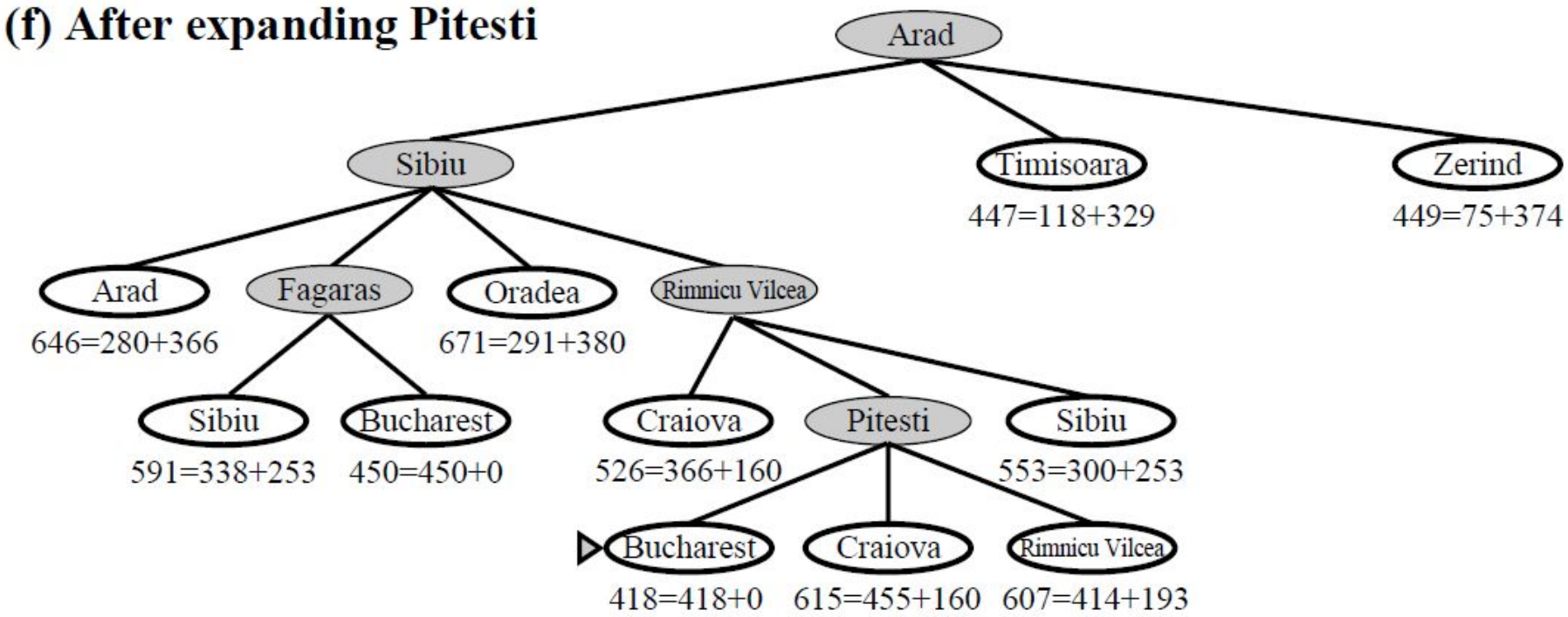
# A* Example - From Arad to Bucharest



# A* Example - From Arad to Bucharest



# A* Example - From Arad to Bucharest



# Admissible Heuristic Functions

## Definition (Admissible Heuristic)

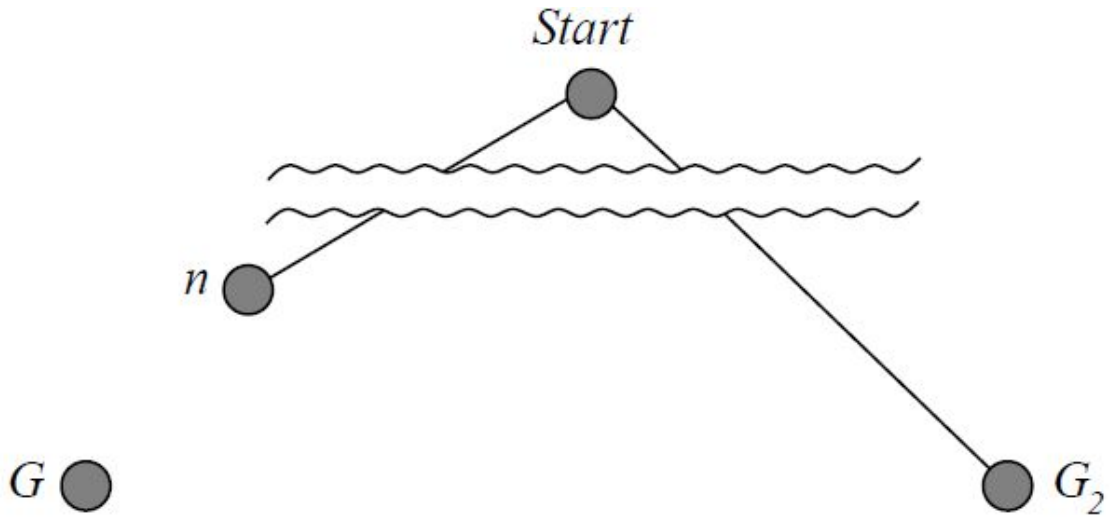
- The perfect heuristic h* assigns every state s the cost of a cheapest path from s to a goal state. A heuristic function **h** is *admissible* if **h**(s) <= h*(s) for all s.

Note: The tree-search version of A∗ is optimal if h(n) is admissible, while the graph-search version is optimal if h(n) is consistent.
If h(n) is consistent, then the values of f(n) along any path are nondecreasing.

# Optimality of A*

Theorem (Optimality of A): Provided that the heuristic function h is admissible, the first solution found by A* is optimal.

Proof: Suppose there exists a goal node G with optimal path cost g*, but A has found another node G2 with g(G2) > g*.



# Optimality of A*

- Let n be a node on the path from the start to G that has not yet been expanded. Since h is admissible, we have

  $f(n) <= g^*$

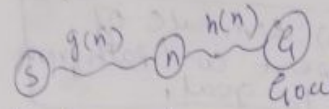
- Since n was not expanded before G2, we have

  $f(G2) <= f(n)$

- and thus $f(G2) <= g^*$. With $h(G2) = 0$, it follows that $g(G2) <= g^*$, in contradiction to the assumption.

## Optimality of A* search Algorithm

A* search is optimal

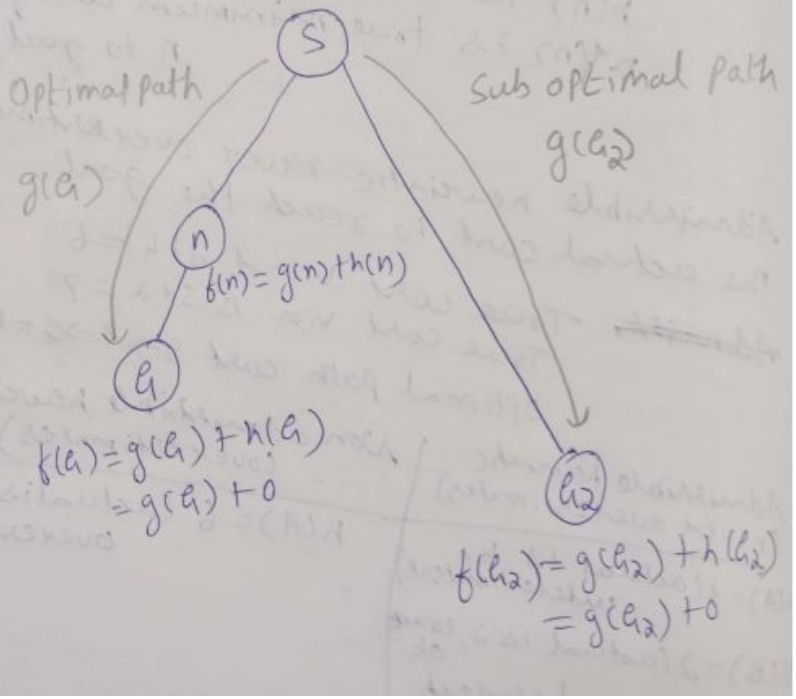



$f(n) = g(n) + h(n)$

actual Estimated cost cost

$g(n)$ = actual cost from S to n

$h(n)$ = estimated cost from n to Goal




→ Let $G$ be an optimal Goal state.

→ Let $G_2$ be an suboptimal Goal state.

$g(G_2) > g(G)$ — (1)

we will show that A* never finds $G_2$ before $G$

we will show this by contradiction.

Suppose A* has selected $G_2$

Consider node 'n' on optimal path to $G$

$f(G) \geq f(n)$ i.e., $f(n) \leq f(G)$

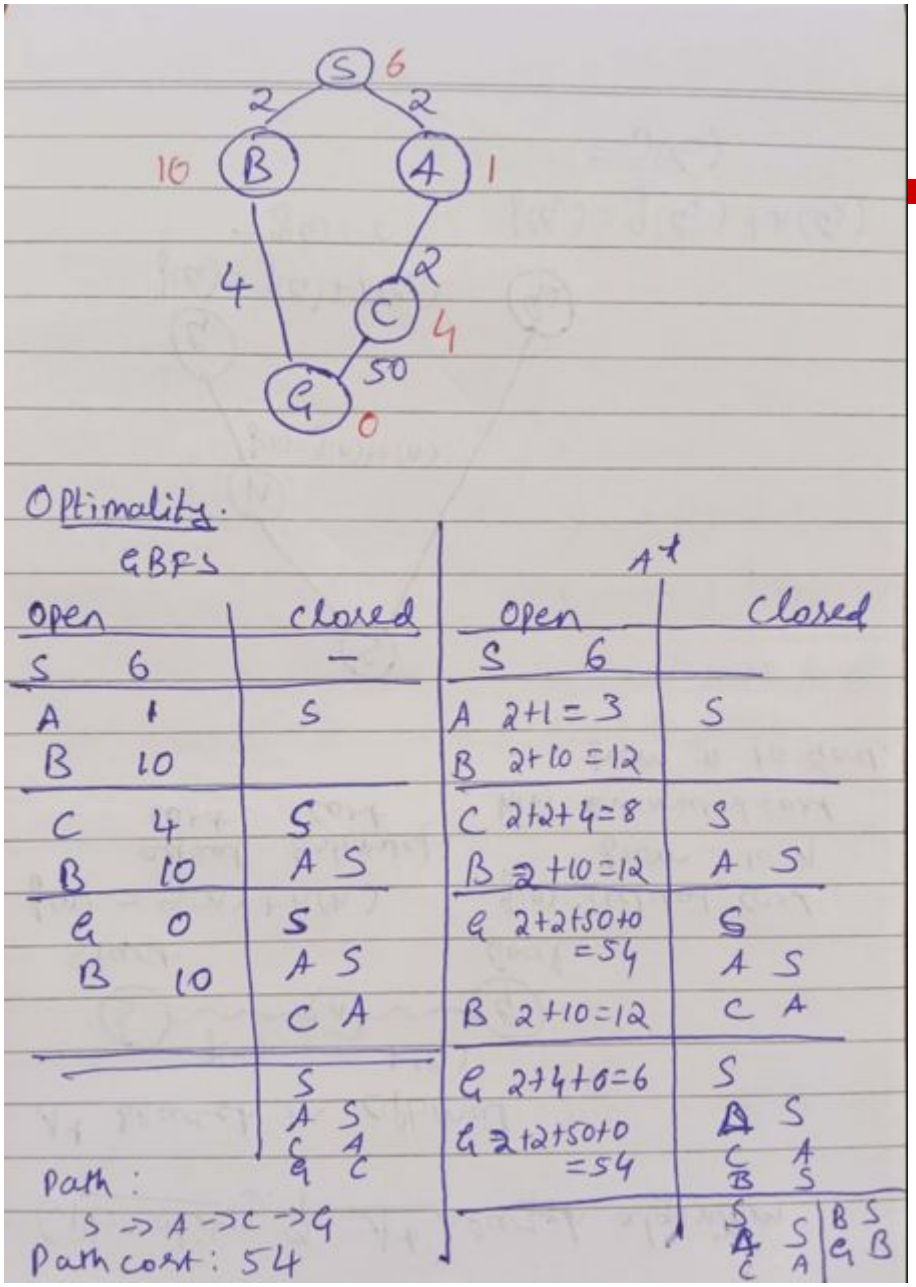
Since 'n' is not choosen for expansion over $G_2$

$f(n) \geq f(G_2)$ i.e, $f(G_2) \leq f(n)$

But, $f(G_2) = g(G_2)$ $\therefore g(G_2) \leq f(n)$

$g(G_2) \leq g(n)$ — (2)

Statement (2) contradicts with (1), hence A* never chooses $G_2$




Optimality.

GBFS

| Open | | Closed |
|---|---|---|
| S | 6 | – |
| A | 1 | S |
| B | 10 | |
| C | 4 | S |
| B | 10 | A S |
| G | 0 | S |
| B | 10 | A S |
| | | C A |
| | | S |
| | | A S |
| | | G A |
| | | C A |

Path:
S → A → C → G

Path cost: 54

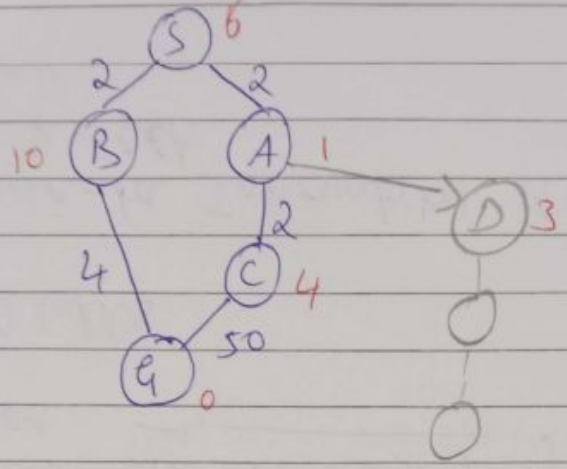
A*

| Open | | Closed |
|---|---|---|
| S | 6 | |
| A | 2+1=3 | S |
| B | 2+10=12 | |
| C | 2+2+4=8 | S |
| B | 2+10=12 | A S |
| G | 2+2+50+0 =54 | G |
| | | A S |
| B | 2+10=12 | C A |
| G | 2+4+0=6 | S |
| G | 2+2+50+0 =54 | A S |
| | | C A |
| | | B S |
| | | S |
| | | A S |
| | | C A |
| | | B S |
| | | G B |

# A* Completeness

Completeness:

- Same as for uniform-cost search: If the branching factor is finite, and every action has cost >= € > 0, then there exists only a finite number of nodes n with f(n) <= g*.



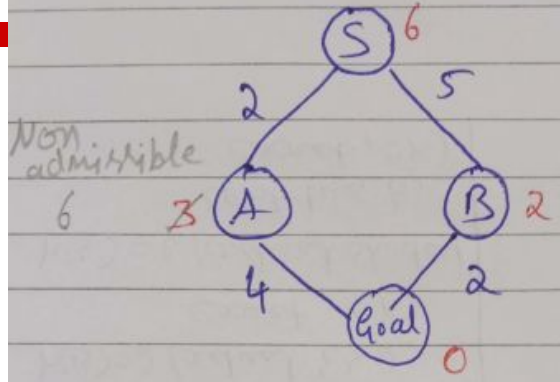

Complete

GBFS: is not Complete if heuristic mislead in infinite or large graphs as it can get stuck chasing the smallest h without considering real cost

A* is Complete if

i) Finite branching factor
ii) Edge costs are positive
iii) Admissible Heuristic: Heuristic function h(n) never overestimates the actual cost to reach to the goal from node n.

A in A* means: A* stands for Admissible




Pathcost: S → B → G: 7

S → A → G: 6

h(n): Estimate of lowest cost path from n to goal.

A heuristic function h(n) is admissible if for every node n

$$h(n) \leq h^*(n)$$

h(n) is estimated cost from n to goal

$h^*(n)$ is true minimum cost from n to goal

Admissible heuristic never overestimates the actual cost to reach the goal.

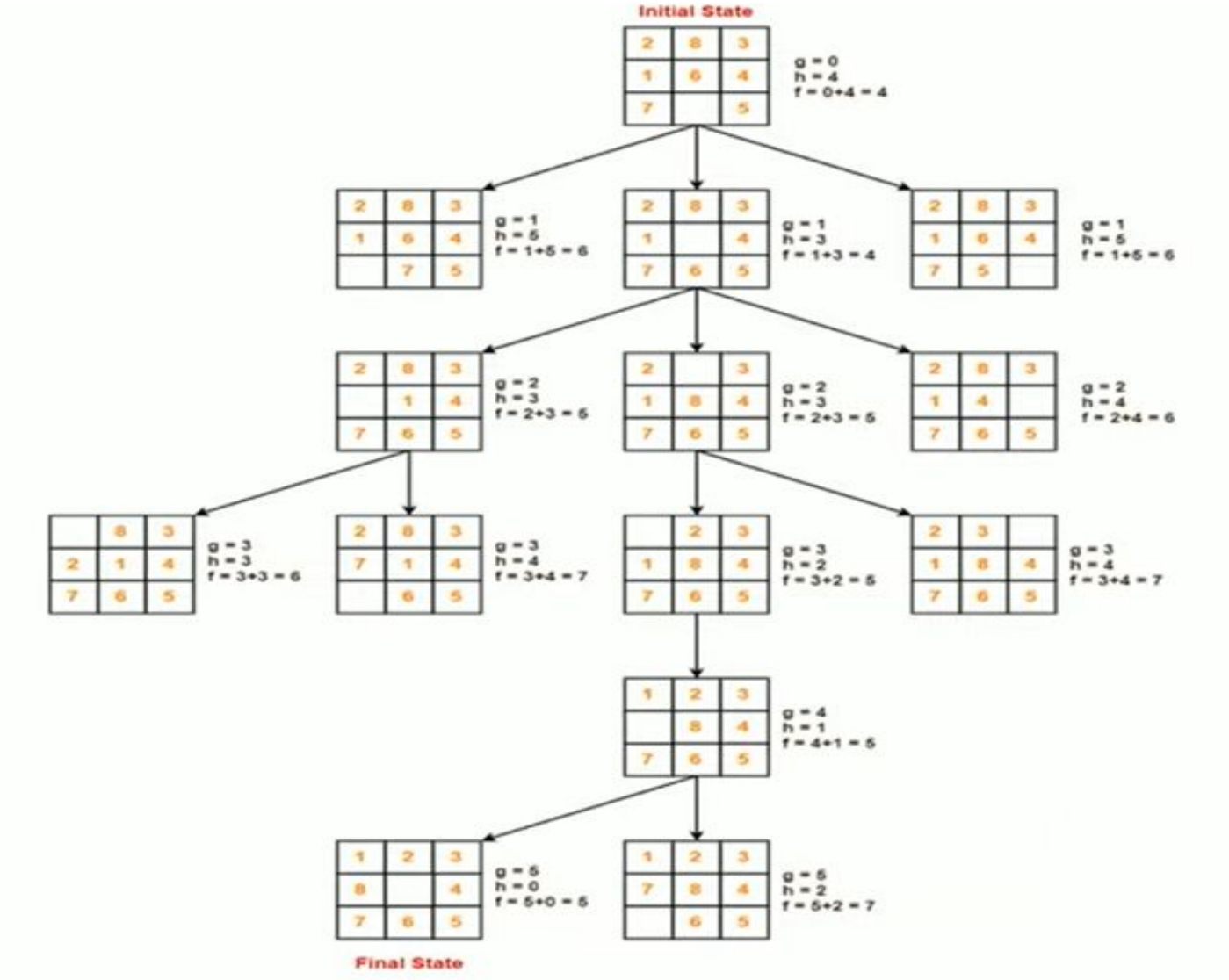
True cost via A $2+4=6$

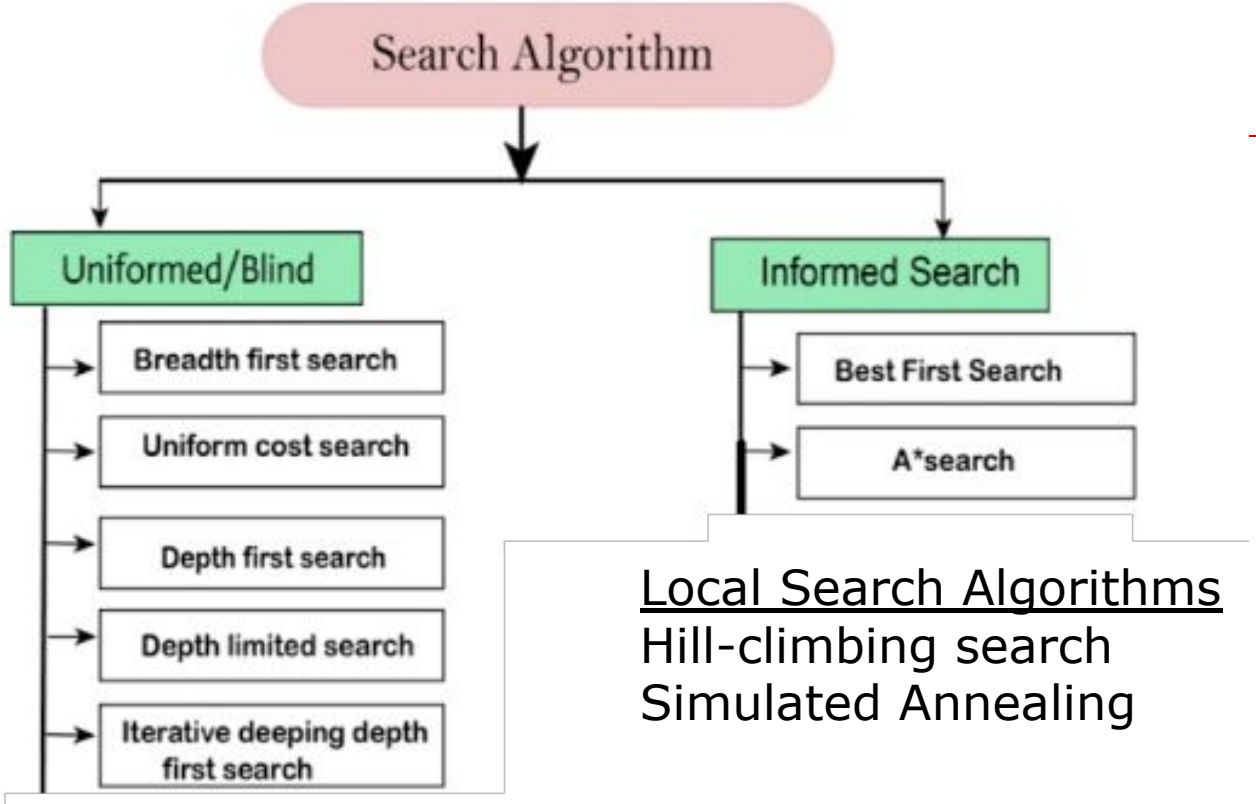
True cost via B $5+2=7$

Optimal path cost $S \rightarrow A \rightarrow G = 6$

| Admissible heuristic (never overestimates) | Non-admissible heuristic (over estimates) |
|---|---|
| $h(A)=3$ (actual is 4 underestimate, OK) | $h(A)=6$ (actual is 4 → overestimate |
| $h(B)=2$ (actual is 2, exact, OK) | |
| $h(S)=6$ (actual Shortest cost to G 6, Exact, OK) | |

# Solution


Initial State
g = 0
h = 4
f = 0+4 = 4
g = 1
h = 5
f = 1+5 = 6
g = 1
h = 3
f = 1+3 = 4
g = 1
h = 5
f = 1+5 = 6
g = 2
h = 3
f = 2+3 = 5
g = 2
h = 3
f = 2+3 = 5
g = 2
h = 4
f = 2+4 = 6
g = 3
h = 3
f = 3+3 = 6
g = 3
h = 4
f = 3+4 = 7
g = 3
h = 2
f = 3+2 = 5
g = 3
h = 4
f = 3+4 = 7
g = 4
h = 1
f = 4+1 = 5
g = 5
h = 0
f = 5+0 = 5
g = 5
h = 2
f = 5+2 = 7
Final State




Local Search Algorithms
Hill-climbing search
Simulated Annealing

# Beyond Classical Search Algorithms

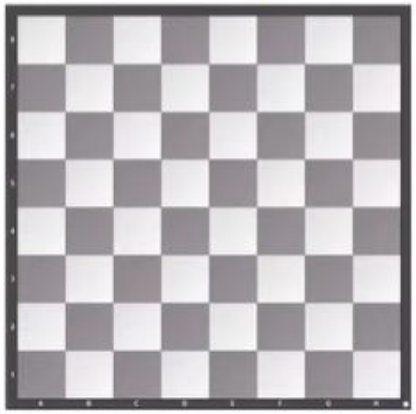
What is Local Search Algorithms?

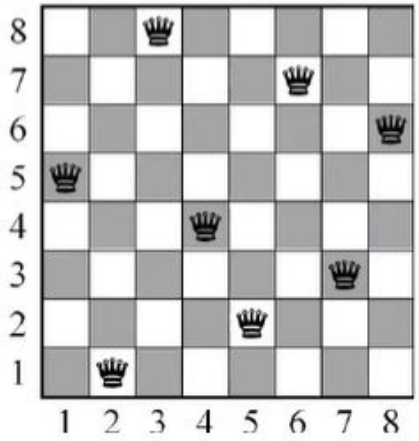
Local Search Algorithms:
Hill Climbing Algorithm
Simulated Annealing

# Local Search Algorithms

# Local Search Algorithms

- The Local search algorithm searches only the final state, not the path to get there.
- For example, in the 8-queens problem,
- we care only about finding a valid final configuration of 8 queens (8 queens arranged on chess board, and no queen can attack other queens) and not the path from initial state to final state.
-





# Local Search Algorithms

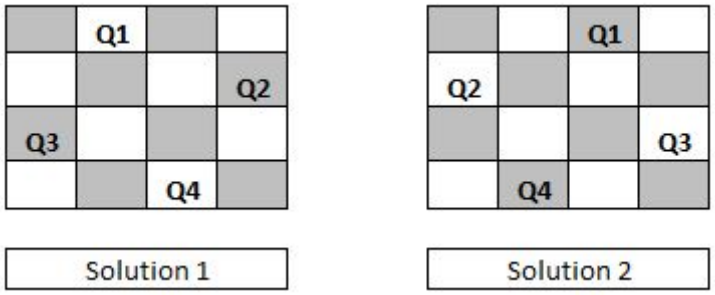
- **Local search** algorithms operate by searching from a start state to neighboring states,
- without keeping track of the paths, nor the set of states that have been reached.
- They are not systematic—
- they might never explore a portion of the search space where a solution actually resides.
- They searches only **the final state**

# What is Local Search Algorithms?

- Local search in the state space, evaluating and modifying one or more current states rather than systematically exploring paths from an initial state.
- These algorithms are suitable for problems in which all that matters is the solution state, not the path cost to reach it.
- The family of local search algorithms includes methods inspired by statistical physics (simulated annealing) and evolutionary biology (genetic algorithms).

# What is N-Queen problem?

- N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal.




4-Queen Problem

# Applications of Local Search Algorithms

- In many problems, however, the path to the goal is irrelevant.
- For example, in the 8-queens problem what matters is the final configuration of queens, not the order in which they are added.
- Other applications include integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming, telecommunications network optimization, vehicle routing, and portfolio management.

# Local Search Algorithms

- Local search algorithms operate using CURRENT NODE a single current node (rather than multiple paths) and generally move only to neighbors of that node. Typically, the paths followed by the search are not retained.
- Although local search algorithms are not systematic, they have two key advantages:

(1) They use **very little memory**—usually a constant amount;

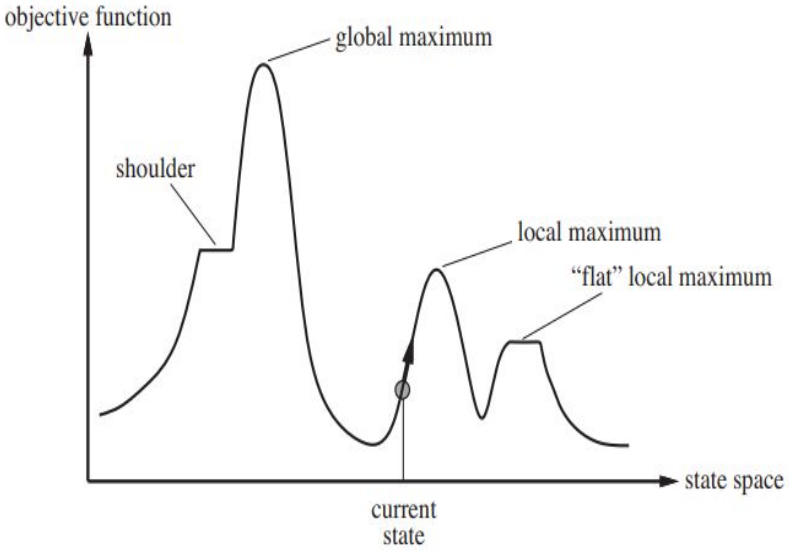
(2) They can often **find reasonable solutions** in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

# Local Search Algorithms

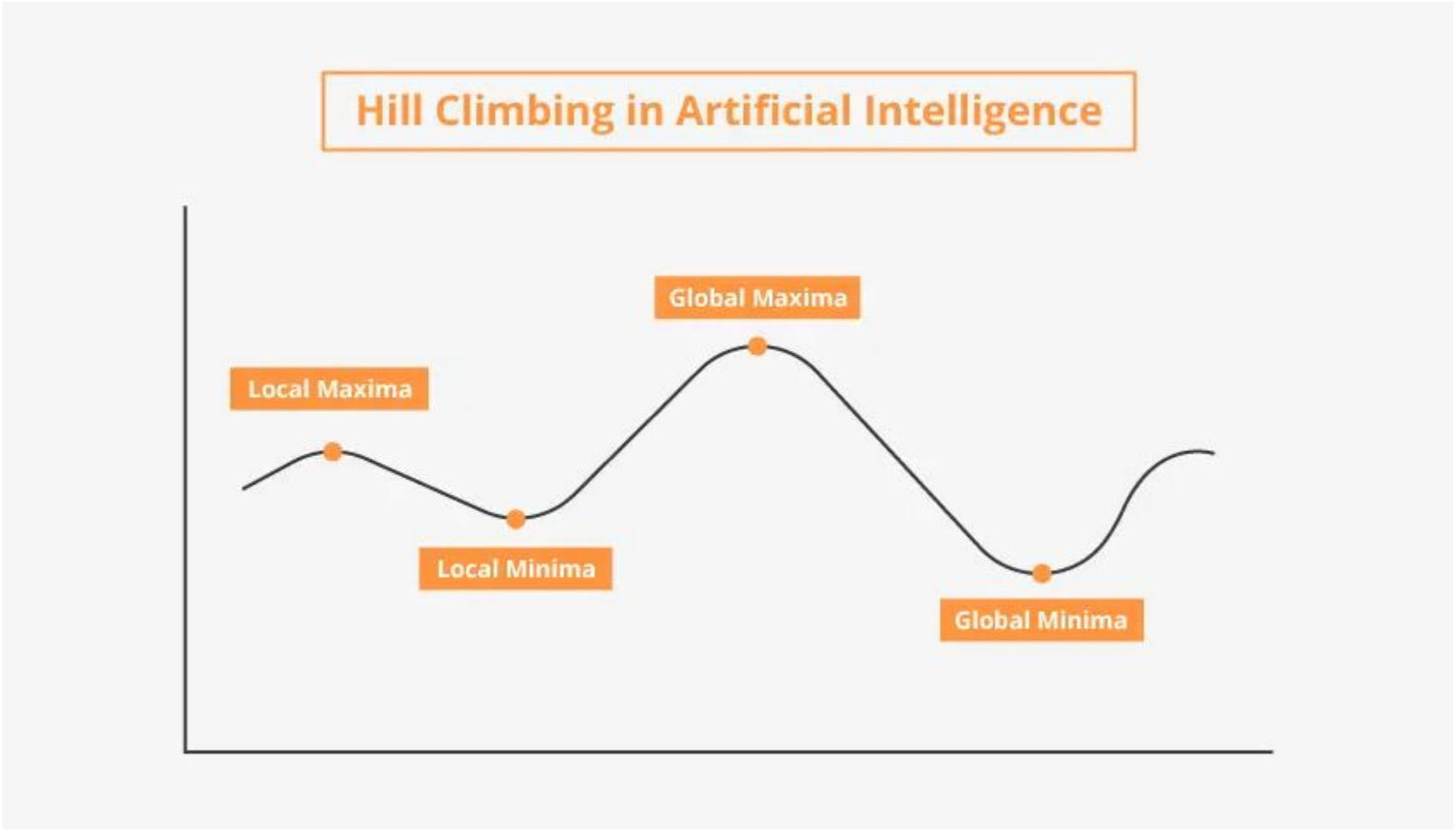
- Local search algorithms are useful for solving pure *optimization problems*, in which the aim is to find the best state according to an **objective function**.

# Understanding Local Search

- State-space landscape:
  - Location: Defined by state
  - Elevation: Defined by the value of the heuristic cost function or objective function

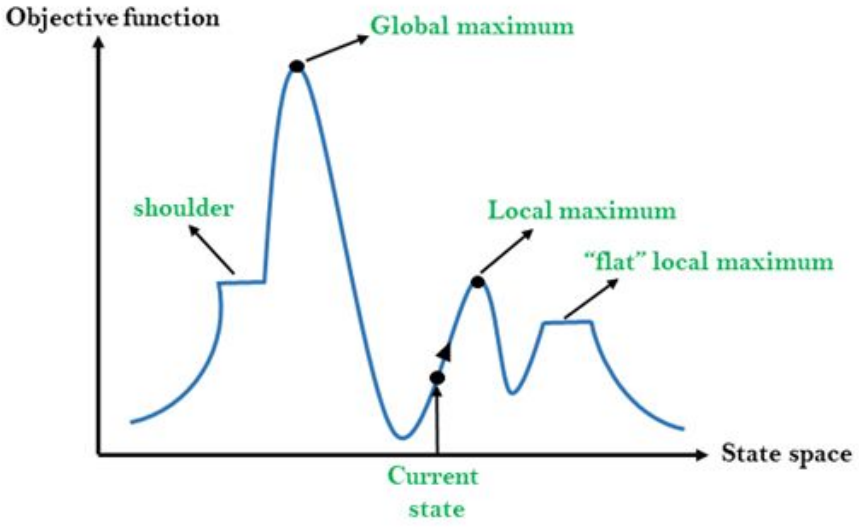



Global minimum:
If elevation corresponds to cost,
then the aim is to find the lowest valley
Global Maximum:
If elevation corresponds to an objective function,
then the aim is to find the highest peak.


Hill Climbing in Artificial Intelligence
Global Maxima
Local Maxima
Local Minima
Global Minima

# State-space Diagram for Hill Climbing:

- The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.
- On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis.
  - If the function on Y-axis is **cost** then, the goal of search is to find the *global minimum* and *local minimum.*
  - If the function of Y-axis is **Objective function**, then the goal of the search is to find the *global maximum* and *local maximum*.



# Hill Climbing Search Algorithm

- Hill climbing is a widely used optimization algorithm in AI that helps find the best possible solution to a given problem.
- It is a form of local search, which means it focuses on finding the optimal solution by making incremental changes to an existing solution and then evaluating whether the new solution is better than the current one.
- The process is analogous to climbing a hill where you continually seek to improve your position until you reach the top, or local maximum, from where no further improvement can be made.

# Hill Climbing Search Algorithm

- Hill climbing is a widely used optimization algorithm in AI that helps find the best possible solution to a given problem.
- It is a form of local search, which means it focuses on finding the optimal solution by making incremental changes to an existing solution and then evaluating whether the new solution is better than the current one.
- The process is analogous to climbing a hill where you continually seek to improve your position until you reach the top, or local maximum, from where no further improvement can be made.

# Basic Concepts of Hill Climbing Algorithms

Hill climbing follows these steps:

1. **Initial State**: Start with an arbitrary or random solution (initial state).
2. **Neighboring States**: Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
3. **Move to Neighbor**: If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.
4. **Termination**: Repeat this process until no neighboring state is better than the current one. At this point, you've reached a local maximum or minimum (depending on whether you're maximizing or minimizing).

# Illustration Hill Clinging to Solve 4-Queens Problem

- State: 4 queens on the board, one per column.
- Successors of a state: All possible states generated by moving a single queen to another square in the same column. So each state has 4 ×3 = 12 successors.
- Heuristic cost function: **h is the number of pairs of queens that are attacking each other**, either directly or indirectly. The global minimum of this function is zero, which occurs only at perfect solutions

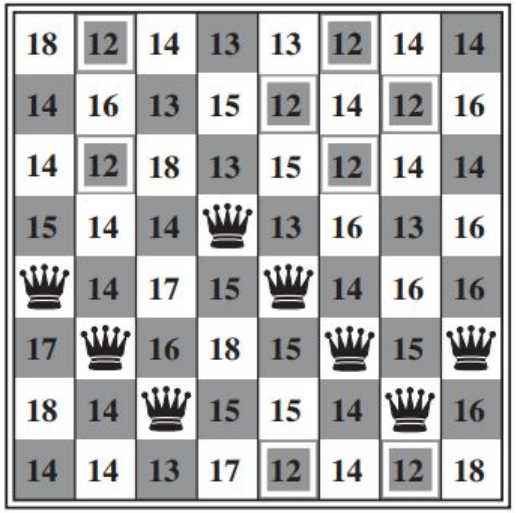
# Question

- What will be the number of Successors for 8-queen problem form current node.

# Question

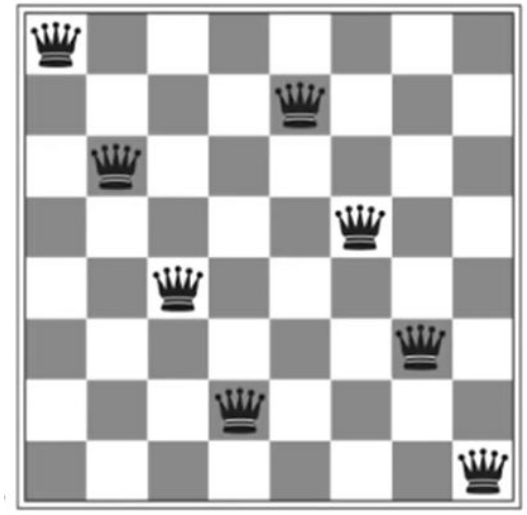
- What will be the number of Successors for 8-queen problem form current node.
- 8x7=Sucessors

- An 8-queens state with heuristic cost estimate h = 17, showing the value of h for each possible successor obtained by moving a queen within its column. The best moves are marked.



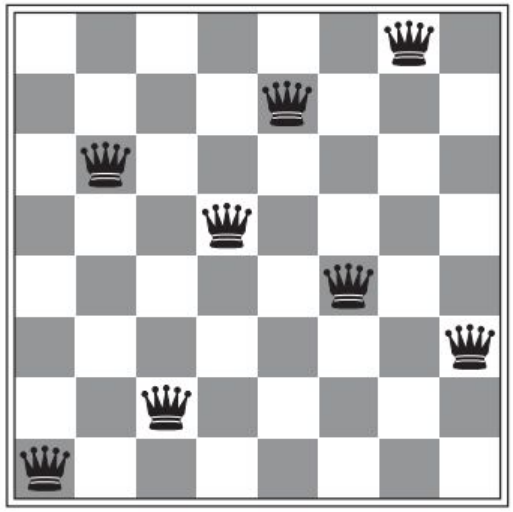
# Question

- What is the h value for this board configuration of 8-Queen problem.
- h=1



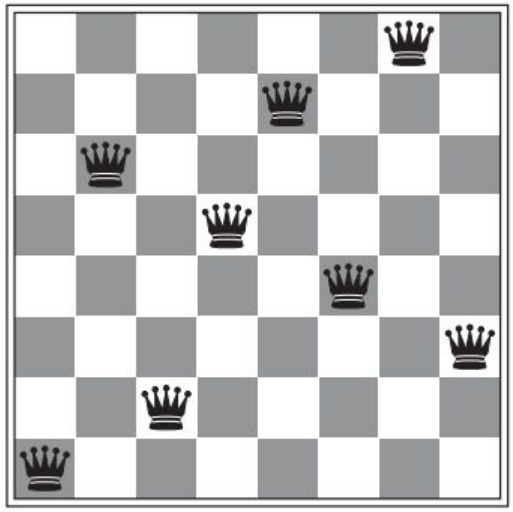
# Question

- What is the h value for this board configuration of 8-Queen problem.



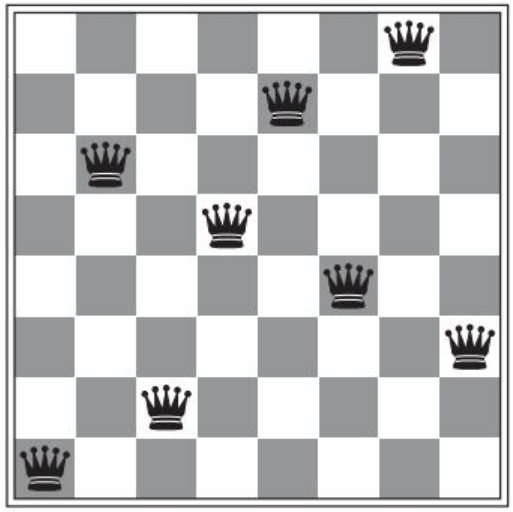
# Question

- What is the h value for this board configuration of 8-Queen problem.

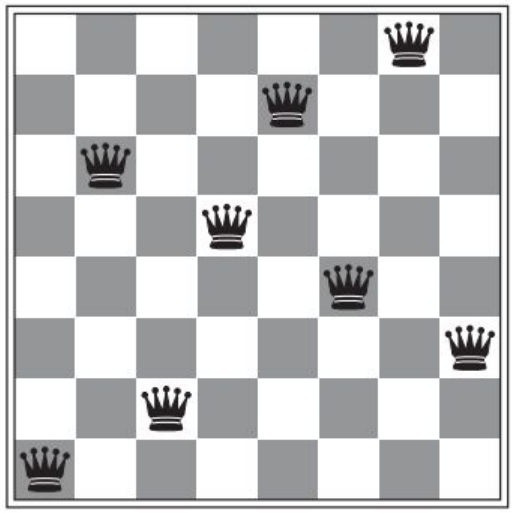


# Question

- What is the h value for this board configuration of 8-Queen problem.
- h=1



- A local minimum in the 8-queens state space; the state has h = 1 but every successor has a higher cost.



# Hill-climbing search algorithm

Hill Climbing:
Trying to find the top of Mount Everest in a thick fog while suffering from amnesia.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

  current ← MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
```

# Hill-climbing search algorithm

- Simply a loop that continually moves in the direction of increasing value—that is, uphill. It terminates when it reaches a “peak” where no neighbor has a higher value.
- The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.
- Hill climbing does not look ahead beyond the immediate neighbors of the current state. This resembles trying to find the top of Mount Everest in a thick fog while suffering from amnesia

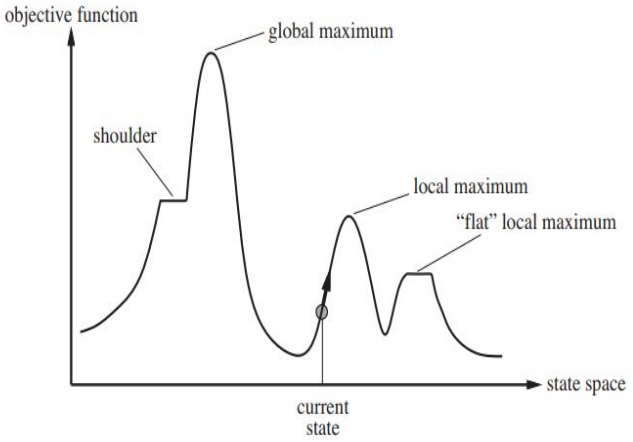
# Hill-climbing search algorithm

Unfortunately, hill climbing often gets stuck for the following reasons:

Local maxima: a local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.

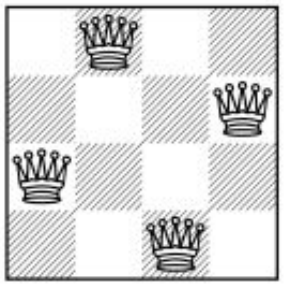
Ridges: Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

Plateaux: Plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which progress is possible.



# The 4-Queens Problem

The 4-queens problem consists of a 4 x 4 chessboard with 4 queens. The goal is to place the 4 queens on the chessboard such that no two queens can attack each other. Each queen attacks anything in the same row, in the same column, or in the same diagonal. The image below gives one solution to this problem.



# Local Search Formulation

## 4-queens problem as a local search problem.

- State: 4 queens on the board. One queen per column.
  - Variables: $x_0, x_1, x_2, x_3$ where $x_i$ is the row position of the queen in column $i$. Assume that there is one queen per column.
  - Domain for each variable: $x_i \in \{0, 1, 2, 3\}, \forall i$.
- Initial state: a random state.
- Goal state: 4 queens on the board. No pair of queens are attacking each other.
- Neighbour relation:
  - Version A: Move a single queen to another square in the same column.
  - Version B: Swap the row positions of two queens.
- Cost function: The number of pairs of queens attacking each other, directly or indirectly.

# Problem

Hill Climbing:

Consider version B of the neighbour relation:
Swap the row positions of two queens.

Question 1:

How many neighbours are there for a state?

# Problem

Hill Climbing:

Consider version B of the neighbour relation:
Swap the row positions of two queens.

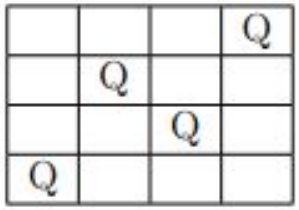
Question 1:

How many neighbours are there for a state?

**Solution:** $\binom{4}{2} = \frac{4 \times 3}{2} = 6$

**Question 2:**

Start with the initial state $x_0 = 3, x_1 = 1, x_2 = 2, x_3 = 0$. Show the steps of executing the hill climbing algorithm until it terminates.




If multiple neighbours have the same cost, choose the neighbour where the pair of queens swapped has the smallest subscript/column number. For example, when we can swap either $(x_0, x_4)$ or $(x_2, x_3)$, we will swap $(x_0, x_4)$. When we can swap either $(x_2, x_3)$ or $(x_2, x_4)$ we will swap $(x_2, x_3)$.

**Solution:** Neighbours of $x_0 = 3, x_1 = 1, x_2 = 2, x_3 = 0$ (cost is 2.)

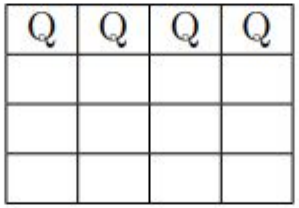
- $x_0 = 1, x_1 = 3, x_2 = 2, x_3 = 0$ cost is 1. Choose this neighbour.
- $x_0 = 2, x_1 = 1, x_2 = 3, x_3 = 0$ cost is 1.
- $x_0 = 0, x_1 = 1, x_2 = 2, x_3 = 3$ cost is 6.
- $x_0 = 3, x_1 = 2, x_2 = 1, x_3 = 0$ cost is 6.
- $x_0 = 3, x_1 = 0, x_2 = 2, x_3 = 1$ cost is 1.
- $x_0 = 3, x_1 = 1, x_2 = 0, x_3 = 2$ cost is 1.

Neighbours of $x_0 = 1, x_1 = 3, x_2 = 2, x_3 = 0$ (cost is 1.)

- $x_0 = 3, x_1 = 1, x_2 = 2, x_3 = 0$ cost is 1.
- $x_0 = 2, x_1 = 3, x_2 = 1, x_3 = 0$ cost is 2.
- $x_0 = 0, x_1 = 3, x_2 = 2, x_3 = 1$ cost is 4.
- $x_0 = 1, x_1 = 2, x_2 = 3, x_3 = 0$ cost is 4.
- $x_0 = 1, x_1 = 0, x_2 = 2, x_3 = 3$ cost is 2.
- $x_0 = 1, x_1 = 3, x_2 = 0, x_3 = 2$ cost is 0. Choose this neighbour.

# Question

Suppose that we are executing the hill climbing algorithm. Let the current state be $x_0 = 0, x_1 = 0, x_2 = 0, x_3 = 0$.




What is the cost of the current state? Is this state a local optimum? If no, give an example of a neighbour with a lower cost. If yes, is this state a global optimum?

Note: Consider version B of the neighbour relation: Swap the row positions of two queens.

**Solution:** The cost of the current state is 6.

Yes, it is a local optimum, but not a global optimum solution.

# Simulated Annealing

# Physical Annealing

- The Simulated Annealing algorithm is based upon Physical Annealing in real life.
- Physical Annealing is the process of heating up a material until it reaches an **annealing temperature** and then
- it will be **cooled down** slowly in order to change the material to a desired structure.
- When the material is hot, the molecular structure is weaker and is more susceptible to change.
- When the material cools down, the molecular structure is harder and is less susceptible to change.





**Algorithm 1** Hill Climbing

```
1: current ← a random state
2: while true do
3:     next ← get-best-neighbour(current)
4:     if cost(current) ≤ cost(next) then
4:         break
5:     end if
6:     current ← next
7: end while
8: return current
```

**Algorithm 2** Simulated Annealing

```
1: current ← initial state
2: T ← a large positive value
3: while T > 0 do
4:     next ← a random neighbour of current
5:     ΔE ← current.cost - next.cost
6:     if ΔE > 0 then
7:         current ← next
8:     else
9:         current ← next with probablity p = e^(ΔE/T)
10:    end if
11:    decrease T
12: end while
13: return current
```

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | | | | |
| 50 | 100 | | | | |
| 50 | 200 | | | | |

$$\Delta E \leftarrow \text{current.cost - next.cost}$$

$$p = e^{\frac{\Delta E}{T}}$$

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | 10 | 1 | 1 | 1 |
| 50 | 100 | -50 | 0.61 | 0.37 | 0.0067 |
| 50 | 200 | -150 | 0.22 | 0.05 | $3.1 \times 10^{-7}$ |

$\Delta E \leftarrow$ current.cost - next.cost

$$p = e^{\frac{\Delta E}{T}}$$

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | 10 | 1 | 1 | 1 |
| 50 | 100 | -50 | 0.61 | 0.37 | 0.0067 |
| 50 | 200 | -150 | 0.22 | 0.05 | $3.1 \times 10^{-7}$ |

**Question:**
Consider a neighbour with a higher cost (i.e. the neighbour is worse than the current state). As the temperature decreases, how does such a worse neighbour change?

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | 10 | 1 | 1 | 1 |
| 50 | 100 | -50 | 0.61 | 0.37 | 0.0067 |
| 50 | 200 | -150 | 0.22 | 0.05 | $3.1 \times 10^{-7}$ |

**Question:**
Consider a neighbour with a higher cost (i.e. the neighbour is worse than the current state). As the temperature decreases, how does such a worse neighbour change?

**Solution:**
As the temperature decreases,
the probability of moving to a neighbour with a higher cost decreases.

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | 10 | 1 | 1 | 1 |
| 50 | 100 | -50 | 0.61 | 0.37 | 0.0067 |
| 50 | 200 | -150 | 0.22 | 0.05 | $3.1 \times 10^{-7}$ |

**Question:**
Consider a neighbour with a higher cost (i.e. the neighbour is worse than the current state). As the difference between the neighbour's cost and the current state's cost increases (i.e. as the neighbour becomes worse), how does the probability of moving to such a worse neighbour change?

**Solution:**
As the neighbour becomes worse,
the probability of moving to a neighbour with a higher cost decreases.

# Simulated Annealing

Suppose that we are running the simulated annealing algorithm. In the following, fill in the values of ΔE and the probabilities of moving to the neighbour under different temperatures.

| current.cost | next.cost | $\Delta E$ | p(T=100) | p(T=50) | p(T=10) |
|---|---|---|---|---|---|
| 50 | 40 | 10 | 1 | 1 | 1 |
| 50 | 100 | -50 | 0.61 | 0.37 | 0.0067 |
| 50 | 200 | -150 | 0.22 | 0.05 | $3.1 \times 10^{-7}$ |

**Question:**
Consider a neighbour with a higher cost (i.e. the neighbour is worse than the current state). As the difference between the neighbour's cost and the current state's cost increases (i.e. as the neighbour becomes worse), how does the probability of moving to such a worse neighbour change?

**Solution:**
As the neighbour becomes worse,
the probability of moving to a neighbour with a higher cost decreases.

# Simulated annealing algorithm

**function** SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  **inputs**: *problem*, a problem
    *schedule*, a mapping from time to "temperature"

  *current* ← MAKE-NODE(*problem*.INITIAL-STATE)
  **for** $t = 1$ **to** $\infty$ **do**
    $T \leftarrow schedule(t)$
    **if** $T = 0$ **then return** *current*
    *next* ← a randomly selected successor of *current*
    $\Delta E \leftarrow next.\text{VALUE} - current.\text{VALUE}$
    **if** $\Delta E > 0$ **then** *current* ← *next*
    **else** *current* ← *next* only with probability $e^{\Delta E/T}$

The simulated annealing algorithm, a version of stochastic hill climbing where some downhill moves are allowed. Downhill moves are accepted readily early in the annealing schedule and then less often as time goes on. The schedule input determines the value of the temperature T as a function of time

# Simulated annealing algorithm

- This algorithm appropriate for nonlinear objective functions where other local search algorithms do not operate well.
- **The simulated-annealing solution is to start by shaking hard (i.e., at a high temperature) and**
- **then gradually reduce the intensity of the shaking (i.e., lower the temperature).**
- Simulated Annealing (SA) is very useful for situations where there are a lot of local minima.

# Difference Between Hill Climbing and Simulated Annealing

| Parameters | Hill Climbing | Simulated Annealing |
|---|---|---|
| Introduction | Hill Climbing is a heuristic optimization process that iteratively advances towards a better solution at each step in order to find the best solution in a given search space. | Simulated Annealing is a probabilistic optimization algorithm that simulates the metallurgical annealing process in order to discover the best solution in a given search area by accepting less-than-ideal solutions with a predetermined probability. |
| Strategy | In order to iteratively move towards the best answer at each stage, Hill Climbing employs a greedy method. It only accepts solutions that are superior to the ones already in place. | Simulated annealing explores the search space and avoids local optimum by employing a probabilistic method to accept a worse solution with a given probability. As the algorithm advances, the likelihood of accepting an inferior answer diminishes. |
| Local vs. Global Optima | Hill Climbing may not locate the global optimum because it is susceptible to becoming caught in local optima. | Simulated annealing has a chance of escaping the local optimum and locating the global optimum. |
| Stopping Criteria | Hill Climbing comes to an end after a certain number of iterations or when it achieves a local optimum. | When the temperature hits a predetermined level or the maximum number of repetitions, simulated annealing comes to an end. |
| Performance | Hill climbing is quick and easy, but it has the potential to become locked in local optima and miss the overall best solution. | Simulated annealing is more efficient at locating the global optimum than Hill Climbing, particularly for complicated situations with numerous local optima. Simulated annealing is slower than Hill Climbing. |

In Iterated Hill Climbing which of the following are true?

- [ ] The algorithm always finds the global optimum
- [ ] The algorithm makes deterministic moves from the current state
- [ ] The algorithm makes stochastic moves from the same start state
- [ ] The algorithm uses different start states chosen randomly
- [ ] None of the above

In Iterated Hill Climbing which of the following are true?

- [ ] The algorithm always finds the global optimum
- [ ] The algorithm makes deterministic moves from the current state
- [ ] The algorithm makes stochastic moves from the same start state
- [ ] The algorithm uses different start states chosen randomly
- [ ] None of the above

**Accepted Answers:**

*The algorithm makes deterministic moves from the current state*

*The algorithm uses different start states chosen randomly*

Consider a state S in the search space for an instance of a problem P. Let moveGen(S) generate k neighbouring states. Which of the following is/are true?

- [ ] Iterated Hill Climbing randomly picks any one of the k neighbours and moves to it.
- [ ] Iterated Hill Climbing always picks the best one of the k neighbours and moves to it
- [ ] Iterated Hill Climbing picks the best one of the k neighbours and moves to it only if it is better than the current state

Consider a state S in the search space for an instance of a problem P. Let moveGen(S) generate k neighbouring states. Which of the following is/are true?

- [ ] Iterated Hill Climbing randomly picks any one of the k neighbours and moves to it.
- [ ] Iterated Hill Climbing always picks the best one of the k neighbours and moves to it
- [ ] Iterated Hill Climbing picks the best one of the k neighbours and moves to it only if it is better than the current state

**Accepted Answers:**

*Iterated Hill Climbing picks the best one of the k neighbours and moves to it only if it is better than the current state*

Which of the following is/are true of Simulated Annealing (SA)?

- [ ] SA generates all neighbours and picks the best one. Then it decides to move to it with a certain probability
- [ ] SA generates only one neighbour randomly and then decides to move to it with a certain probability
- [ ] SA generates one neighbour randomly and always moves to it if it is better
- [ ] SA generates one neighbour randomly and may move to it even if it is worse

Which of the following is/are true of Simulated Annealing (SA)?

- [ ] SA generates all neighbours and picks the best one. Then it decides to move to it with a certain probability
- [ ] SA generates only one neighbour randomly and then decides to move to it with a certain probability
- [ ] SA generates one neighbour randomly and always moves to it if it is better
- [ ] SA generates one neighbour randomly and may move to it even if it is worse

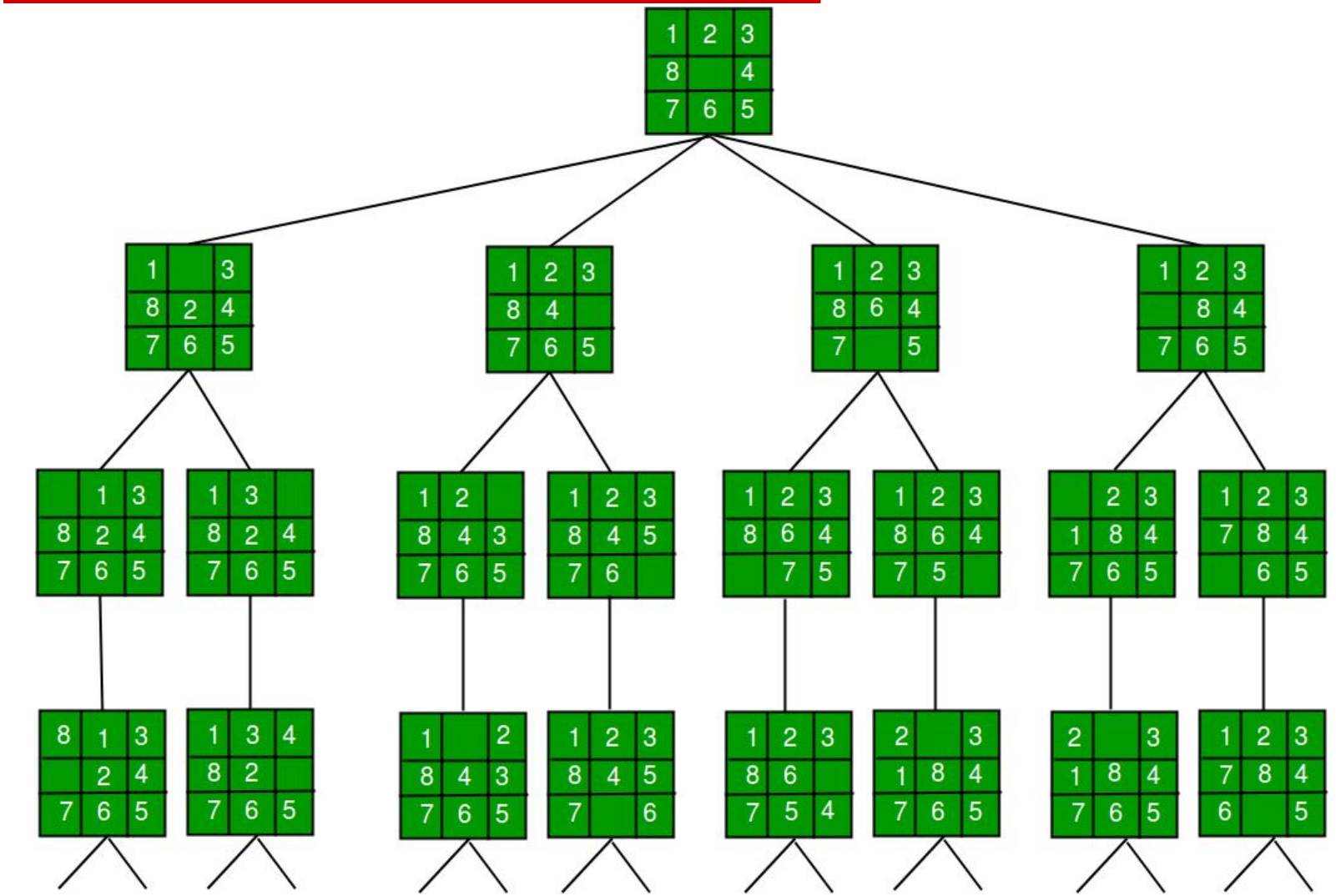
**Accepted Answers:**

*SA generates only one neighbour randomly and then decides to move to it with a certain probability*

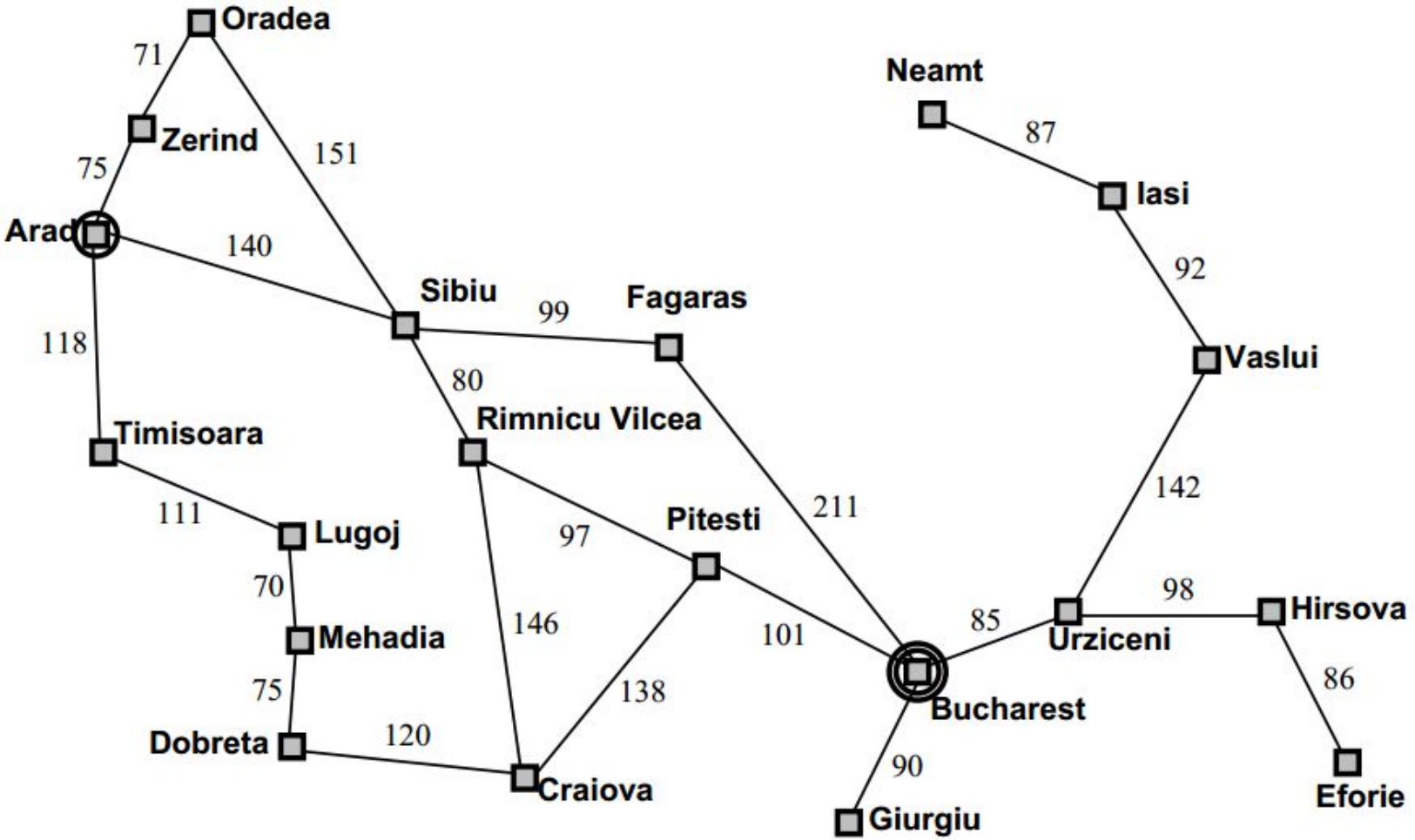
*SA generates one neighbour randomly and may move to it even if it is worse*

# Thanks for Listening

# 8 Puzzle – Partial State Space



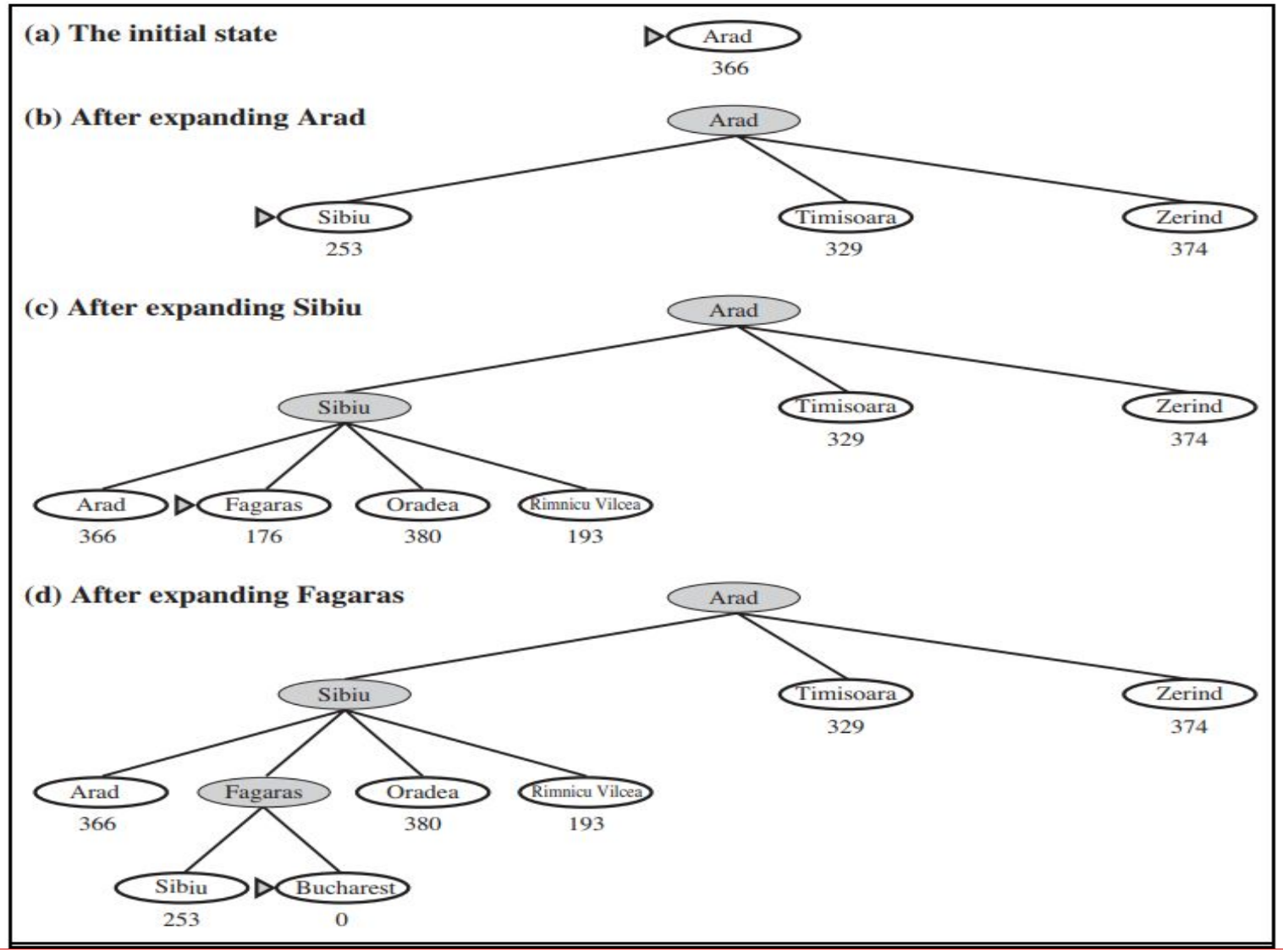
# Example – Romania Route Map



| | | | |
|---|---|---|---|
| **Arad** | 366 | **Mehadia** | 241 |
| **Bucharest** | 0 | **Neamt** | 234 |
| **Craiova** | 160 | **Oradea** | 380 |
| **Drobeta** | 242 | **Pitesti** | 100 |
| **Eforie** | 161 | **Rimnicu Vilcea** | 193 |
| **Fagaras** | 176 | **Sibiu** | 253 |
| **Giurgiu** | 77 | **Timisoara** | 329 |
| **Hirsova** | 151 | **Urziceni** | 80 |
| **Iasi** | 226 | **Vaslui** | 199 |
| **Lugoj** | 244 | **Zerind** | 374 |

**Figure 3.22** Values of $h_{SLD}$—straight-line distances to Bucharest.

Stages in a greedy best-first tree search for Bucharest with the straight-line distance heuristic hSLD . Nodes are labeled with their h-values.



# Questionnaire

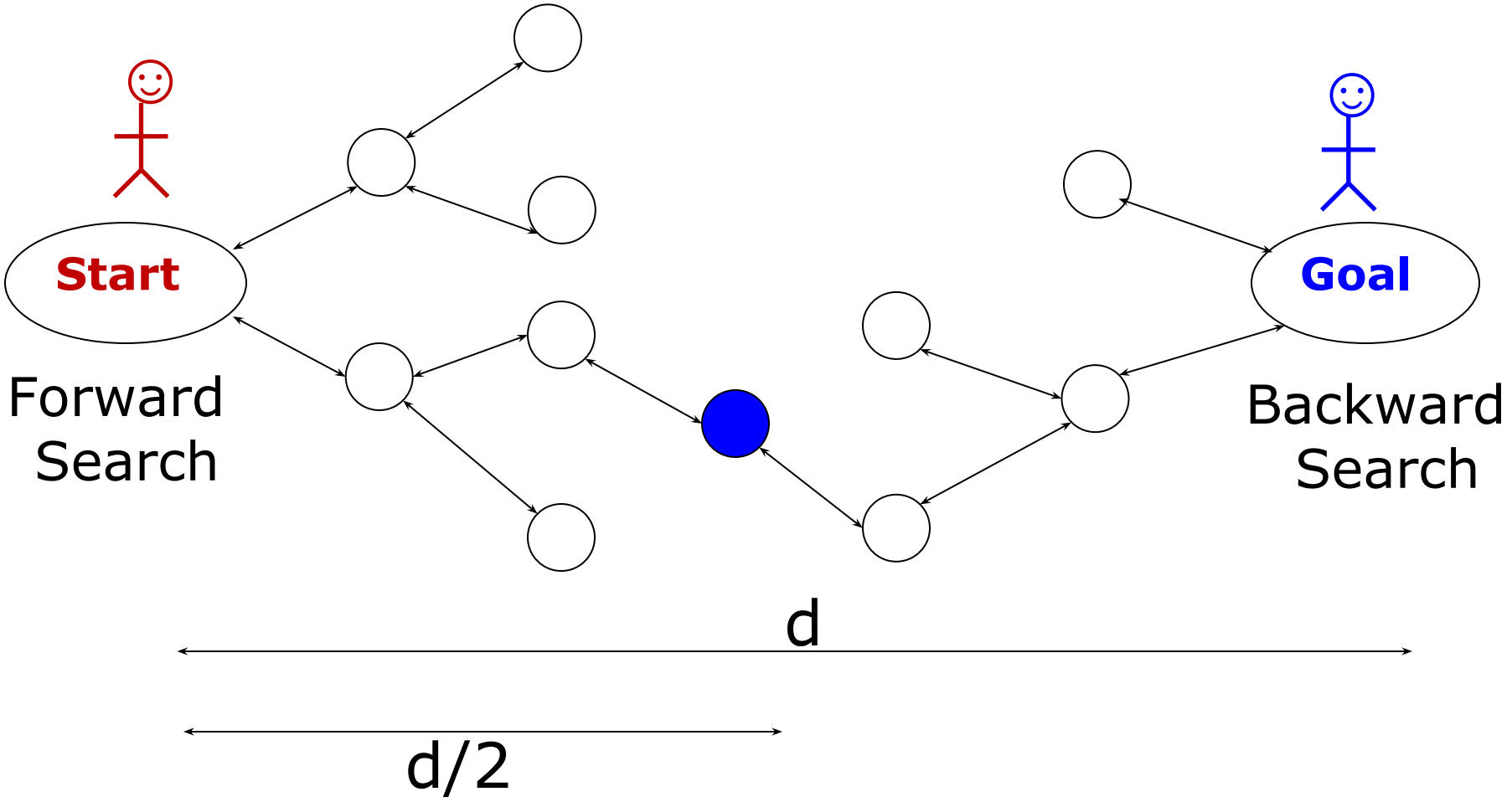
If wet set h(n) := 0 for all n, what does A become?

1. Breadth-first search
2. Uniform-cost search
3. Depth-first search
4. Depth-limited search

If wet set h(n) := 0 for all n, what does greedy best-rst search become?

1. Breadth-first search
2. Uniform-cost search
3. Depth-first search
4. Depth-limited search

# Bidirectional Search



# Questionnaire

Which are “Problems”?

- You didn’t understand any of this.
- Your vacuum cleaner wants to clean your apartment.
- Your bus today will probably be late.
- You want to win a Chess game.

# Homework : Water Jug Problem

You have three jugs measuring 12 gallons, 8 gallons and 3 gallons and a water faucet. You need to measure out exactly one gallon.

Initial State: All three jugs are empty

Goal test: Some jug contains exactly one gallon

Successor function: Applying the

**Action** *transfer* to jugs i and j with capacities Ci and Cj and containing Gi and Gj gallons of water, respectively, leaves jug i with max(0, Gi−(Cj −Gj)) gallons of water and jug j with min(Cj, Gi+ Gj) gallons of water.

Applying the **action** *fill* to jug i leaves it with Ci gallons of water.

Cost function: Charge one point for each gallon of water transferred and each gallon of water filled

Question: Show the representation of State Space and solution. Is it possible to get all 3 jugs with 1 gallon water?